

Lab Manual
for
R PROGRAMMING LAB
(DS504PC)
Academic Year: 2024-2025
III B. TECH I SEMESTER (R22 AUTONOMOUS)



DEPARTMENT OF CSE (Data Science)



ACE
Engineering College
An Autonomous Institution



Ghatkesar, Hyderabad - 501 301, Telangana.
Approved by AICTE & Affiliated to JNTUH
NBA Accredited B.Tech Courses, Accorded NAAC A-Grade with 3.20 CGPA

INDEX

S.No	Contents	Page No.
1	Department Vision, Mission, POs and PSOs	1
2	Objectives and Course Outcomes of the lab	3
3	List of Experiments	4
4	Introduction to Lab	6
5	Algorithms/Programs with Viva questions	7
6	Additional experiments- beyond syllabus	70
7	Scheme of Evaluation	71
8	References	72

Institute Vision:

To be a leading Technical Institute to prepare high quality Engineers to cater the needs of the stakeholders in the field of Engineering and Technology with global competence fundamental comprehensive analytical skills, critical reasoning, research aptitude, entrepreneur skills, ethical values and social concern.

Institute Mission:

Imparting Quality Technical Education to young Engineers by providing the state-of-the-art laboratories, quality instructions by qualified and experienced faculty and research facilities to meet the requirements of stakeholders in real time usage and in training them to excel in competitive examinations for higher education and employment to interface globally emerging techno-informative challenges in the growth corridor of techno-excellence.

Department Vision:

To produce excellent standard, quality education of professionals by imparting cognitive learning environment, research and industrial orientation to become innovative Data Science Professional.

Department Mission:

M1: To develop professionals in the areas of math's (probability and statistics, liner algebra and Calculus), natural language processing, text mining, and problem solving.

M2: To educate the students with latest technologies to update their knowledge in the field of AI and Data science.

M3: To impart quality and value-based education and contribute towards the innovation of computing system, data science to raise satisfaction level of all stakeholders.

M4: Enabling students to get expertise in critical skills with data science education and facilitate socially responsive research and innovation

M5: Our effort is to apply new advancements in high performance computing hardware and software.

Program Outcomes:

Program Outcomes	Statement
PO1	An ability to apply knowledge of mathematics, science, and engineering and knowledge of Fundamental Principles.
PO2	An ability to Identify, formulate and solve engineering problems.
PO3	An ability to design a system, component, or process to meet desired needs in Computer Science and Engineering within realistic constraints such as economic, environmental, social, political, ethical, health and safety, manufacturability and sustainability, Design and Modeling.
PO4	An ability to design and conduct experiments, as well as to analyze and interpret data, Experimentation & Interpret/Engineering Analysis.
PO5	An ability to use the techniques, skills and modern Computer Science and Engineering tools necessary for system design with embedded engineering practice.
PO6	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	The broad education necessary to understand the impact of engineering solutions in a global, economic, environmental, and societal context.
PO8	An understanding of professional and ethical responsibility.
PO9	An ability to function on multidisciplinary teams.
PO10	An ability to communicate effectively.
PO11	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	A recognition of the need for, and an ability to engage in life-long learning.

Course Objectives:

Familiarize with R basic programming concepts, various data structures for handling datasets, various graph representations and Exploratory Data Analysis concepts.

Course Outcomes:

CO	Course Outcomes
CO1	Setup R programming environment.
CO2	Understand and use R–Data types and R–Data Structures.
CO3	Develop programming logic using R–Packages.
CO4	Analyze data set susing R–programming capabilities

Requirements:

- Intel based desktop PCs LAN CONNECTED with minimum of 166 MHZ or faster processor with atleast 64 MB RAM and 100 MB free disk space.
- S/ws: R Base and RStudio

List of Experiments:

S. No.	Program Name
1.	Download and install R-Programming environment and install basic package using install.packages() command in R.
2.	Learn all the basics of R-Programming (Data types, Variables, Operators etc.,)
3.	Write R command to i. Illustrate summation, subtraction, multiplication, and division operations on vectors using vectors. ii. Enumerate multiplication and division operations between matrices and vectors in R console
4.	Write R command to i) Illustrates the usage of Vector subsetting and Matrix subsetting ii) Write a program to create an array of 3×3 matrices with 3 rows and 3 columns.
5.	Write an R program to draw i) Pie chart ii) 3D Pie Chart, iii) Bar Chart along with chart legend by considering suitable CSV file
6.	Create a CSV file having Speed and Distance attributes with 1000 records. Write R program to draw i) Box plots ii) Histogram iii) Line Graph iv) Multiple line graphs v) Scatterplot To demonstrate the relation between the cars speed and the distance.
7.	Implement different data structures in R (Vectors, Lists, DataFrames)
8.	Write an R program to read a csv file and analyze the data in the file using EDA (Exploratory Data Analysis) techniques.
9.	Write an R program to illustrate Linear Regression and Multilinear Regression considering suitable CSV file
10.	What is the purpose of the apply() function in R? Can you explain how it works and provide an example?

Module Wise Outcomes:

S.No.	Program Name	Course Outcome
1	Download and install R-Programming environment and install basic package using install.packages() command in R.	Setup R programming environment.
2	Learn all the basics of R-Programming (Data types, Variables, Operators etc.,)	Understand and use R-Data types and R-Data Structures.
3	Write R command to i) Illustrate summation, subtraction, multiplication, and division operations on vectors using vectors. ii) Enumerate multiplication and division operations between matrices and vectors in R console	Understand and use R-Data types and R-Data Structures.
4	Write R command to iii) Illustrates the usage of Vector subsetting and Matrix subsetting iv) Write a program to create an array of 3x3 matrices with 3 rows and 3 columns.	Understand and use R-Data types and R-Data Structures.
5	Write an R program to draw i) Pie chart ii) 3D Pie Chart, iii) Bar Chart along with chart legend by considering suitable CSV file	Analyze data set using R-programming capabilities
6	Create a CSV file having Speed and Distance attributes with 1000 records. Write R program to draw i. Box plots, ii. Histogram, iii. Line Graph, iv. Multiple line graphs v. Scatterplot. To demonstrate the relation between the cars speed and the distance.	Analyze data set using R-programming capabilities
7	Implement different data structures in R (Vectors, Lists, DataFrames)	Analyze data set using R-programming capabilities
8	Write an R program to read a csv file and analyze the data in the file using EDA (Exploratory Data Analysis) techniques.	Analyze data set using R-programming capabilities
9	Write an R program to illustrate Linear Regression and Multilinear Regression considering suitable CSV file	Understand and use R-Data types and R-Data Structures.
10	What is the purpose of the apply() function in R? Can you explain how it works and provide an example?	Analyze data set using R-programming capabilities

Introduction to Lab:

R is a programming language and environment primarily designed for statistical computing and graphics. It provides a wide variety of statistical and graphical techniques, and it is highly extensible. Here's a detailed overview of R programming:

History and Development: R was initially created by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand, in the early 1990s. It is an implementation of the S programming language, which was developed at Bell Laboratories by John Chambers and colleagues in the late 1970s.

Features:

- Open Source: R is free and open-source, which means anyone can download and modify its source code.*
- Cross-Platform Compatibility: R runs on multiple operating systems, including Windows, macOS, and Linux.*
- Rich Ecosystem: R has a vast ecosystem of packages contributed by users worldwide. These packages extend R's functionality in various domains, including statistics, machine learning, data visualization, and more.*
- Data Handling: R provides powerful data handling capabilities, allowing users to import, manipulate, clean, and analyze data from various sources.*
- Graphics: R offers extensive tools for creating high-quality graphics and visualizations, enabling users to explore data and communicate findings effectively.*
- Statistical Analysis: R is widely used for statistical analysis, offering a comprehensive set of functions and algorithms for descriptive statistics, inferential statistics, hypothesis testing, regression analysis, time series analysis, and more.*
- Open Source: R is free and open-source, which means anyone can download and modify its source code.*
- Cross-Platform Compatibility: R runs on multiple operating systems, including Windows, macOS, and Linux.*
- Rich Ecosystem: R has a vast ecosystem of packages contributed by users worldwide. These packages extend R's functionality in various domains, including statistics, machine learning, data visualization, and more.*
- Data Handling: R provides powerful data handling capabilities, allowing users to import, manipulate, clean, and analyze data from various sources.*
- Graphics: R offers extensive tools for creating high-quality graphics and visualizations, enabling users to explore data and communicate findings effectively.*

- *Statistical Analysis: R is widely used for statistical analysis, offering a comprehensive set of functions and algorithms for descriptive statistics, inferential statistics, hypothesis testing, regression analysis, time series analysis, and more.*

Syntax:

- *R syntax is relatively straightforward and expressive, making it accessible to both beginners and experienced programmers.*
- *Functions and operators are central to R programming.*
- *R uses vectorized operations, allowing users to perform operations on entire vectors or matrices without explicit looping.*
- *R supports both procedural and functional programming paradigms.*

Data Types:

- *R supports various data types, including numeric, character, logical, integer, complex, and factors.*
- *Data structures such as vectors, matrices, arrays, lists, and data frames are fundamental in R for organizing and manipulating data.*

Control Structures:

- *R provides control structures like loops (for, while) and conditional statements (if-else, switch) for flow control and decision-making.*

Functions:

- *Functions are essential building blocks in R programming.*
- *Users can define their functions using the function() keyword.*
- *R functions can accept arguments, return values, and have default parameter values.*

Data Structures in R:

- *Vectors: One-dimensional arrays that can hold numeric, character, or logical data.*
- *Matrices: Two-dimensional arrays with rows and columns.*
- *Arrays: Multi-dimensional generalization of matrices.*
- *Data Frames: Tabular data structures similar to tables in a relational database.*
- *Lists: Ordered collections of objects, which can be of different types.*

Control Structures:

- *Loops: for, while, and repeat loops are used for repetitive tasks.*

- *Conditional Statements: if, else if, and else statements for decision making.*

Packages:

- *R's functionality can be extended through packages, which are collections of R functions, data, and documentation.*
- *The Comprehensive R Archive Network (CRAN) hosts thousands of R packages contributed by users.*
- *Users can install packages using the `install.packages()` function and load them into their R environment using the `library()` function.*

Integrated Development Environment (IDE):

- *RStudio is the most popular integrated development environment (IDE) for R programming. It provides a user-friendly interface for writing, executing, and debugging R code.*
- *Other IDEs include Emacs, Vim, and Visual Studio Code with R extensions.*

Applications of R:

Data Analysis and Visualization: R is widely used in various fields such as finance, healthcare, genetics, and social sciences for analyzing and visualizing data.

Statistical Modeling and Machine Learning: R provides a wide range of statistical modeling and machine learning techniques for predictive analytics and data mining.

Bioinformatics: R is extensively used in bioinformatics for analyzing biological data, sequencing data, and genomic data.

Economics and Finance: R is used in econometrics and financial modeling for analyzing economic data and forecasting financial markets.

Lab Programs with Viva Questions:

Program 1: Download and install R-Programming environment and install basic package using `install.packages()` command in R.

AIM: How to install R and RStudio in Windows and Ubuntu operating systems

DESCRIPTION:

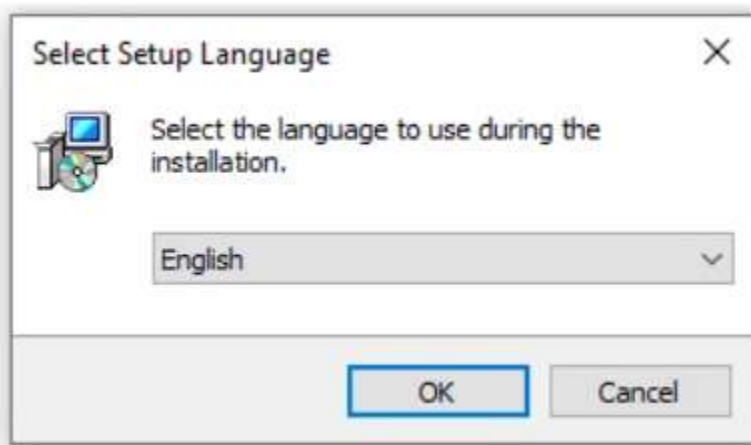
R and RStudio are two different programs that work together. R is the programming language and RStudio is this is a front end program that lets you write R code, view plots, etc. Each program must be downloaded separately.

Download and install R-Programming environment

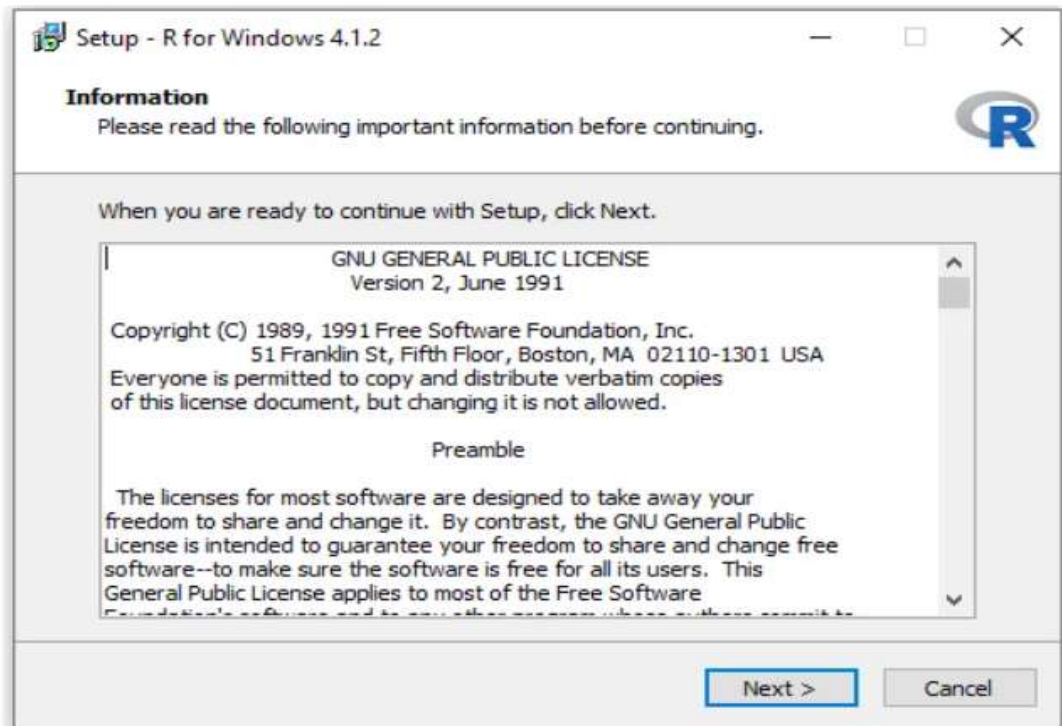
Installing R on Windows OS

To install R on Windows OS:

1. Go to the [CRAN](https://cran.r-project.org/) website.
2. Click on "**Download R for Windows**".
3. Click on "**install R for the first time**" link to download the R executable (.exe) file.
4. Run the R executable file to start installation, and allow the app to make changes to your device.
5. Select the installation language.



6. Follow the installation instructions.



7. Click on **"Finish"** to exit the installation setup.



R has now been successfully installed on your Windows OS. Open the R GUI to start writing R codes.

Installing R on MacOS X

Installing R on MacOS X is very similar to installing R on Window OS. The difference is the file format that you have to download. The procedure is as follows:

1. Go to the [CRAN](https://cran.r-project.org/) website.
2. Click on "**Download R for macOS**".
3. Download the latest version of the R GUI under (.pkg file) under "**Latest release**". You can download much older versions by following the "old directory" or "CRAN archive" links.
4. Run the .pkg file, and follow the installation instructions.

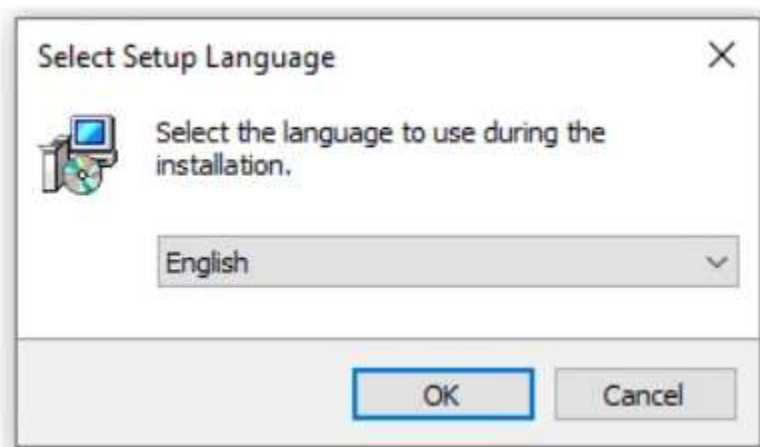
Additional R interfaces

Other than the R GUI, the other ways to interface with R include RStudio Integrated Development Environment (**RStudio IDE**) and **Jupyter Notebook**. To run R on RStudio, you first need to install R on your computer, while to run R on Jupyter Notebook, you need to install an R kernel. RStudio and Jupyter Notebook provide an interactive and friendly graphical interface to R that greatly improves users' experience.

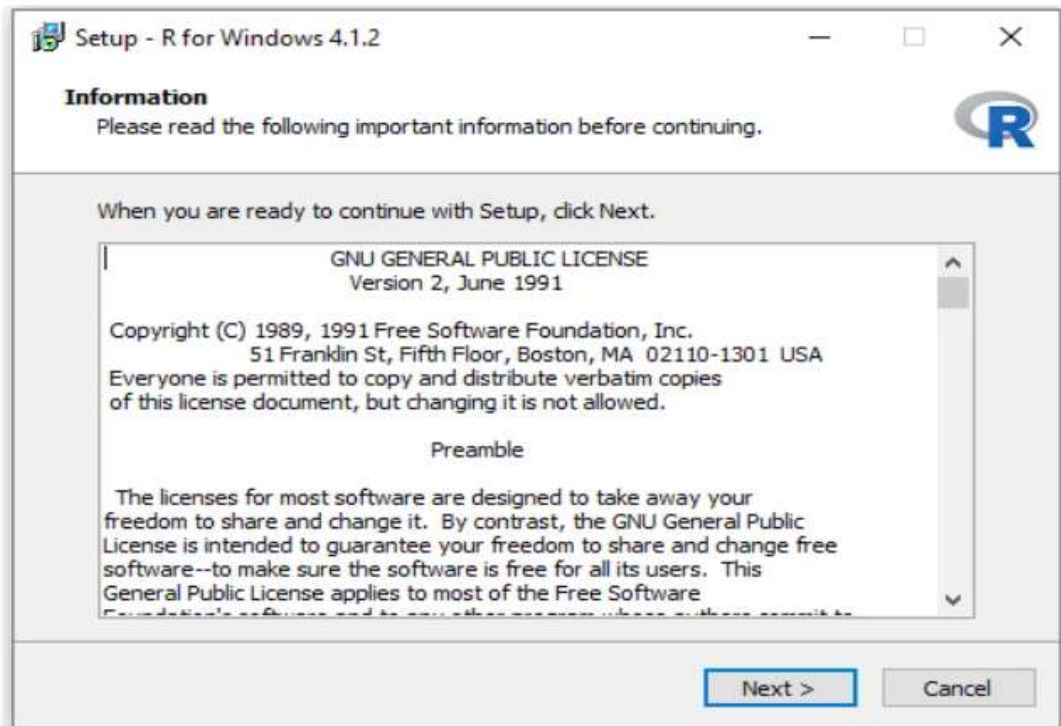
Installing RStudio Desktop

To install RStudio Desktop on your computer, do the following:

1. Go to the [RStudio](https://www.rstudio.com/) website.
2. Click on "**DOWNLOAD**" in the top-right corner.
3. Click on "**DOWNLOAD**" under the "**RStudio Open Source License**".
4. Download RStudio Desktop recommended for your computer.
5. Run the RStudio Executable file (.exe) for Windows OS or the Apple Image Disk file (.dmg) for macOS X.



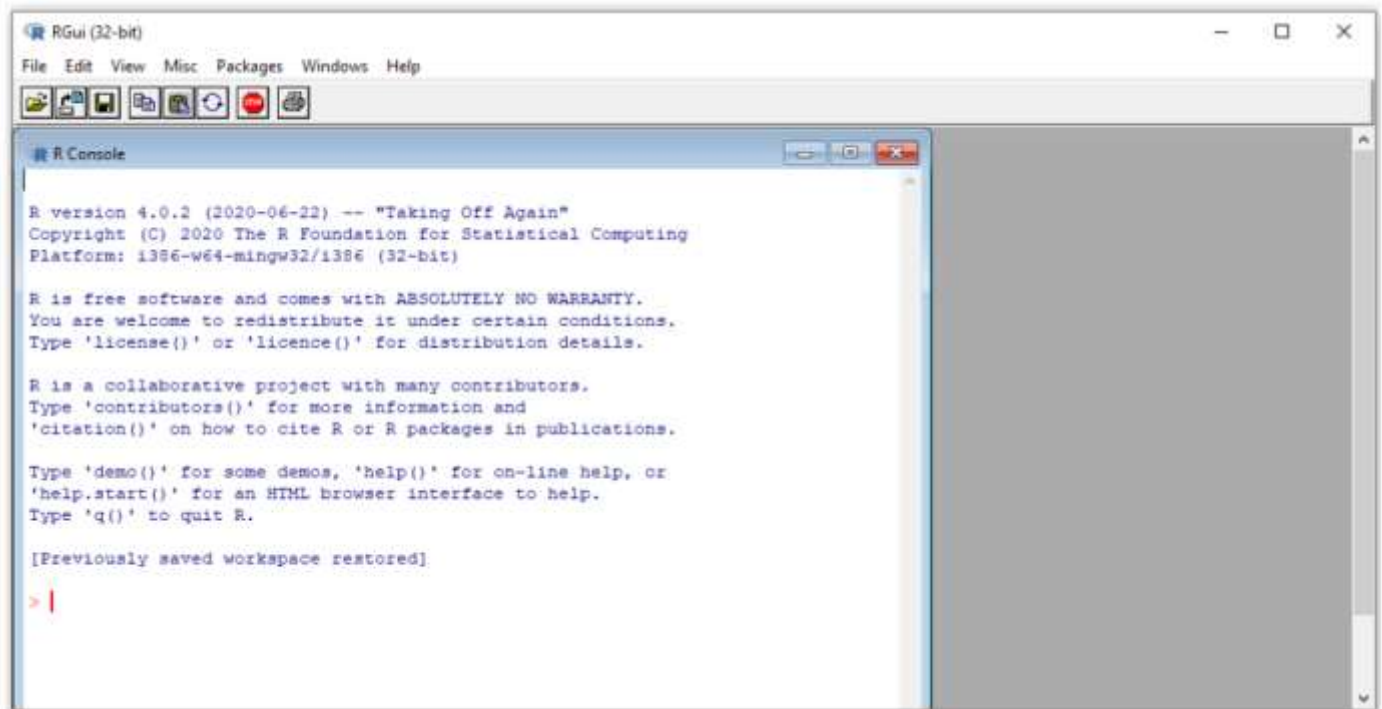
6. Follow the installation instructions to complete RStudio Desktop installation.



RStudio is now successfully installed on your computer. The RStudio Desktop IDE interface is shown in the figure below:



Another way to interface with R using RStudio is with the *RStudio Server*. RStudio Server provides a browser-based R interface.



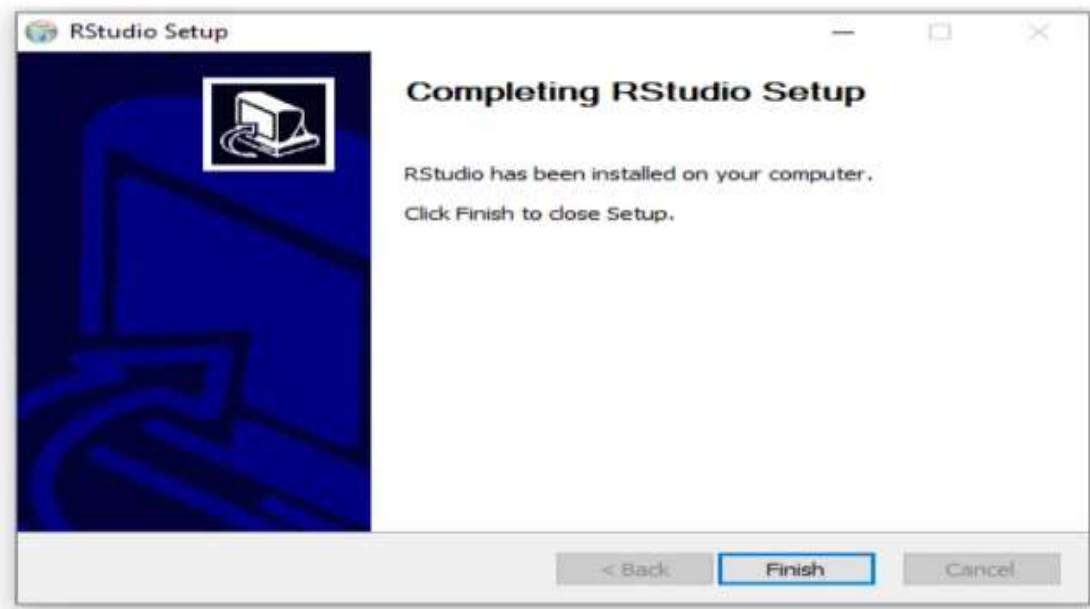
Installing RStudio Desktop

To install RStudio Desktop on your computer, do the following:

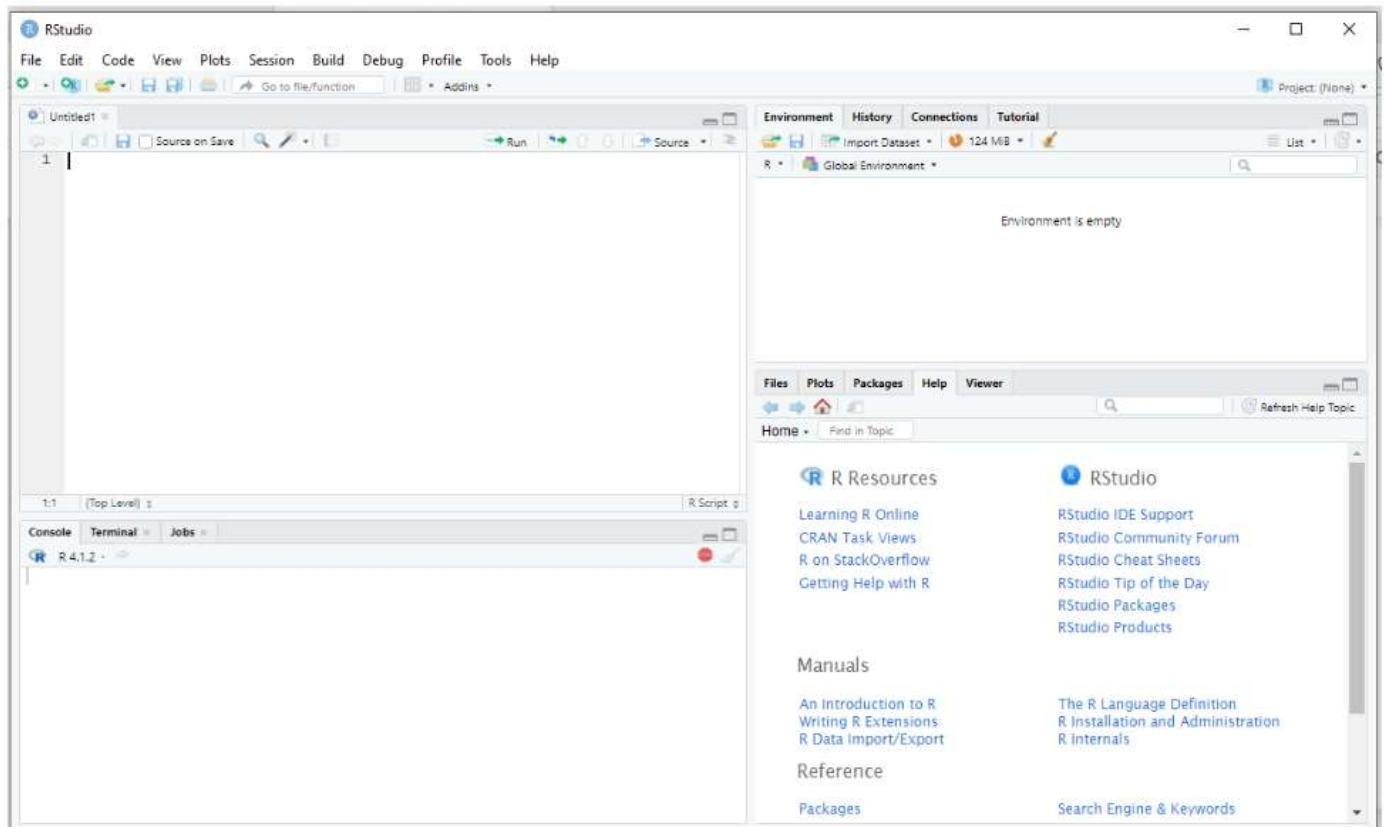
1. Go to the [RStudio](https://www.rstudio.com/) website.
2. Click on **"DOWNLOAD"** in the top-right corner.
3. Click on **"DOWNLOAD"** under the **"RStudio Open Source License"**.
4. Download RStudio Desktop recommended for your computer.
5. Run the RStudio Executable file (.exe) for Windows OS or the Apple Image Disk file (.dmg) for macOS X.



6. Follow the installation instructions to complete RStudio Desktop installation.



RStudio is now successfully installed on your computer. The RStudio Desktop IDE interface is shown in the figure below:



Another way to interface with R using RStudio is with the *RStudio Server*. RStudio Server provides a browser-based R interface.

Installing R Studio on Ubuntu

Installing R Studio on Ubuntu has steps similar to Windows:

Through Terminal

Step 1: Open terminal (Ctrl+Alt+T) in Ubuntu.

Step 2: Update the package's cache.

```
sudo apt-get update
```

Step 3: Install R environment.

```
sudo apt -y install r-base
```

Check for the version of R package using

```
R --version
```

Step 4: Check R installation by using the following command.

```
user@Ubuntu:~$ R
```

(Note that R version should be 3.6+ to be able to install all packages like tm, e1071, etc.). If there is issue with R version, see the end of the post.

Step 5: Exit the terminal.

Installation of packages in R

To install packages in R using the `install.packages()` command, you simply need to provide the name of the package(s) you want to install as an argument to the function. Here's how you can do it:

Install a single package

```
install.packages("package_name")
```

```
install.packages(c("package1", "package2", "package3")) # Install multiple packages
```

Replace "**package_name**", "**package1**", "**package2**", and "**package3**" with the names of the packages you want to install. Make sure to use the exact name of the package.

For example, if you want to install the **ggplot2** package, you can do:

```
install.packages("ggplot2")
```

This will install the **ggplot2** package along with its dependencies.

After the installation is complete, you can load the package into your R session using the **library()** function:

Load the installed package

```
library(ggplot2)
```

Now you can use the functions and features provided by the "ggplot2" package in your R session.

VIVA QUESTIONS & ANSWERS

1. What is R?

R is a programming language and environment specifically designed for statistical computing and graphics.

2. What is RStudio?

Studio is an integrated development environment (IDE) for R. It provides a user-friendly interface for writing, executing, and debugging R code.

3. What are the main components of the RStudio interface?

The main components of the RStudio interface include the script editor, console, workspace, environment/history, file viewer, and plots panes.

4. How do you install packages in R?

You can install packages in R using the `install.packages()` function. For example:
`install.packages("package_name")`.

5. How do you load packages in R?

You can load packages in R using the `library()` function. For example: `library(package_name)`.

6. What are some common data structures in R?

Common data structures in R include vectors, matrices, lists, data frames, and factors.

7. What is the use of the `apply()` family of functions in R?

The `apply()` family of functions in R is used to apply a function to the rows or columns of matrices, arrays, or data frames.

8. How do you generate random numbers in R?

You can generate random numbers in R using functions like `runif()` for uniform distribution, `rnorm()` for normal distribution, and `sample()` for sampling from a vector.

9. What is the purpose of `ggplot2` in R?

`ggplot2` is a popular package in R used for data visualization. It provides a powerful and flexible system for creating elegant and complex plots.

10. How do you save plots created in R?

You can save plots created in R using the `ggsave()` function for `ggplot2` plots or the `pdf()`, `png()`, `jpeg()`, or `svg()` functions for base R plots.

11. What is the difference between a histogram and a bar plot in R?

A histogram is used to represent the distribution of continuous variables, while a bar plot is used to represent the distribution of categorical variables.

Program 2: Learn all the basics of R-Programming (Data types, Variables, Operators etc.,)

AIM: Learning the basics of R-Programming (Data types, Variables, Operators etc.,)

DESCRIPTION:

Basics of R Syntax:

R syntax is generally straightforward and similar to other programming languages.

Statements in R are typically ended with a semicolon ;, but it's optional in most cases.

Comments can be added using the # symbol.

Data Structures in R:

Vectors: One-dimensional arrays that can hold elements of the same data type.

Matrices: Two-dimensional arrays with elements arranged in rows and columns.

Lists: Collections of objects that can be of different data types.

Data Frames: Two-dimensional structures similar to matrices but with columns that can contain different data types.

Factors: Used to represent categorical data.

Control Structures in R:

Conditional statements: if, else if, else.

Looping structures: for loops, while loops.

Functions:

Functions allow you to encapsulate a piece of code that can be reused.

You can define your own functions using the function() keyword.

Packages:

R has a vast ecosystem of packages that extend its functionality.

You can install packages using the install.packages() function and load them using the library() function.

Data Import and Export:

R provides functions to import data from various formats like CSV, Excel, databases, etc.

Common functions include read.csv(), read.table(), read.xlsx(), etc.

Data can be exported using functions like write.csv(), write.table(), etc.

Variable in R Programming:

Variables are used to store data values. R is dynamically typed, meaning you don't need to declare the data type explicitly.

- Assignment Operators:
 - `<-` (preferred in R): `x <- 10`
 - `=` (also valid): `x = 10`
- Variable Naming Rules:
 - Must start with a letter.
 - Can contain letters, digits, and underscores (`_`).
 - Case-sensitive (`MyVar` is different from `myvar`).

Example:

```
x <- 42      # Assigning a number
y <- "Hello" # Assigning a string
z <- TRUE    # Assigning a Logical value
```

Reserved Keywords in R

Following are the reserved keywords in R

For	In	repeat	while	function
If	else	Next	break	TRUE
FALSE	NULL	Inf	NaN	NA
NA_integer_	NA_real_	NA_complex_	NA_character_	...

NA:– Not Available is used to represent missing values.

NULL:– It represents a missing or an undefined value.

NaN:– It is a short form for Not a Number(eg:- 0/0).

TRUE/FALSE: – These are used to represent Logical values.

Inf :– It denotes Infinity(eg:- 1/0).

If else, repeat, while, function, for, in, next, and break:– These are Used as looping statements, conditional statements, and functions.

NA_integer_, NA_real_, NA_complex_, and NA_character _:– These represent missing values of other atomic types.

For example:

- `x = 15` implicitly assigns a numeric data type to the variable 'x'.
- `mystring = "Hello, World!"`

Constants in R

The entities whose values are fixed are called constants.

There are two types of constants in R:

- **Numeric Constants:** All numeric values such as integer, double, or complex fall under this category. Numeric constants followed by 'L' and 'i' are considered as integer and complex respectively.
- **Character Constants:** These constants are represented by single (') or double (") quotes called delimiters.

Data Types:

A [variable](#) can store different types of values such as numbers, characters etc. These different types of data that we can use in our code are called **data types**. For example,

```
x <- 123L
```

Here, 123L is an integer data. So the data type of the variable x is integer.

We can verify this by printing the class of x.

```
x <- 123L
# print value of x
print(x)
# print type of x
print(class(x))
```

Output

```
[1] 123
[1] "integer"
```

Here, x is a variable of data type integer.

Different Types of Data Types

R has several basic data types, each serving a specific purpose.

Data Type	Description	Examples
Numeric	Real numbers (default type for numbers in R).	2.5, 100
Integer	Whole numbers (explicitly declared with L).	5L, 100L
Logical	Boolean values.	TRUE, FALSE
Character	Strings of text.	"R Programming", "Data"
Complex	Complex numbers.	1 + 4i
Raw	Represents raw bytes.	Not commonly used.

Let's discuss each of these R data types one by one.

1. Logical Data Type

The `logical` data type in R is also known as **boolean** data type. It can only have two values: `TRUE` and `FALSE`. For example,

```
bool1 <- TRUE

print(bool1)
print(class(bool1))

bool2 <- FALSE

print(bool2)
print(class(bool2))
```

Output

```
[1] TRUE
[1] "logical"
[1] FALSE
[1] "logical"
```

In the above example,

- `bool1` has the value `TRUE`,
- `bool2` has the value `FALSE`.

Here, we get `"logical"` when we check the type of both variables.

Note: You can also define logical variables with a single letter - `T` for `TRUE` or `F` for `FALSE`. For example,

```
is_weekend <- F
print(class(is_weekend)) # "logical"
```

2. Numeric Data Type

In R, the `numeric` data type represents all real numbers with or without decimal values. For example,

```
# floating point values
weight <- 63.5

print(weight)
print(class(weight))

# real numbers
height <- 182

print(height)
```

```
print(class(height))
```

Output

```
[1] 63.5  
[1] "numeric"  
[1] 182  
[1] "numeric"
```

Here, both `weight` and `height` are variables of `numeric` type.

3. Integer Data Type

The `integer` data type specifies real values without decimal points. We use the suffix `L` to specify integer data. For example,

```
integer_variable <- 186L  
print(class(integer_variable))
```

Output

```
[1] "integer"
```

Here, `186L` is an integer data. So we get `"integer"` when we print the class of `integer_variable`.

4. Complex Data Type

The `complex` data type is used to specify purely imaginary values in R. We use the suffix `i` to specify the imaginary part. For example,

```
# 2i represents imaginary part  
complex_value <- 3 + 2i  
# print class of complex_value  
print(class(complex_value))
```

Output

```
[1] "complex"
```

Here, `3 + 2i` is of `complex` data type because it has an imaginary part `2i`.

5. Character Data Type

The `character` data type is used to specify character or string values in a variable.

In programming, a string is a set of characters. For example, `'A'` is a single character and `"Apple"` is a string.

You can use single quotes `"` or double quotes `"` to represent strings. In general, we use:

- `"` for character variables
- `"` for string variables

For example,

```
# create a string variable
fruit <- "Apple"

print(class(fruit))

# create a character variable
my_char <- 'A'

print(class(my_char))
```

Output

```
[1] "character"
[1] "character"
```

Here, both the variables - `fruit` and `my_char` - are of `character` data type.

6. Raw Data Type

A `raw` data type specifies values as raw bytes. You can use the following methods to convert character data types to a raw data type and vice-versa:

- `charToRaw()` - converts character data to raw data
- `rawToChar()` - converts raw data to character data

For example,

```
# convert character to raw
raw_variable <- charToRaw("Welcome to Programiz")

print(raw_variable)
print(class(raw_variable))

# convert raw to character
char_variable <- rawToChar(raw_variable)

print(char_variable)
print(class(char_variable))
```

Output

```
[1] 57 65 6c 63 6f 6d 65 20 74 6f 20 50 72 6f 67 72 61 6d 69 7a
[1] "raw"
[1] "Welcome to Programiz"
[1] "character"
```

In this program,

- We have first used the `charToRaw()` function to convert the string "Welcome to Programiz" to raw bytes.

This is why we get "raw" as output when we print the class of `raw_variable`.

- Then, we have used the `rawToChar()` function to convert the data in `raw_variable` back to character form.

This is why we get "character" as output when we print the class of `char_variable`.

R Operators

In this article, you will learn about different R operators with the help of examples.

R has many operators to carry out different mathematical and logical operations. Operators perform tasks including arithmetic, logical and bitwise operations.

Type of operators in R

Operators in R can mainly be classified into the following categories:

- Arithmetic Operators: +, -, *, /, %%, %/%, ^
- Relational Operators: <, <=, >, >=, ==, !=
- Logical Operators: &, |, !, &&, ||
- Assignment Operators: =, <-, <<-, ->, ->>
- Miscellaneous operators: :, %in%, %*%

Arithmetic Operators:

Operator	Description	Example	Result
+	Addition	7 + 3	10
-	Subtraction	7 - 3	4
*	Multiplication	7 * 3	21
/	Division	7 / 3	2.3333
%%	Modulus	7 %% 3	1
%/%	Integer Division	7 %/% 3	2

Operator	Description	Example	Result
<code>^</code> or <code>**</code>	Exponentiation	2^3	8

Let's look at an example illustrating the use of the above operators:

```
x <- 5
y <- 16
x + y
x - y
x * y
y / x
y %/% x
y %% x
y ^ x
```

Output

```
[1] 21
[1] -11
[1] 80
[1] 3.2
[1] 3
[1] 1
[1] 1048576
```

Relational (Comparison) Operators:

Operator	Description	Example	Result
<code>==</code>	Equal to	$7 == 3$	FALSE
<code>!=</code>	Not equal to	$7 != 3$	TRUE
<code><</code>	Less than	$7 < 3$	FALSE
<code>></code>	Greater than	$7 > 3$	TRUE
<code><=</code>	Less than or equal to	$7 <= 3$	FALSE
<code>>=</code>	Greater than or equal to	$7 >= 3$	TRUE

Logical Operators:

Operator	Description
!	Logical NOT
&	Element-wise logical AND
&&	Logical AND
	Element-wise logical OR
	Logical OR

Logical operators are used to carry out Boolean operations like **AND**, **OR** etc.

Operators **&** and **|** perform element-wise operation producing result having length of the longer operand.

But **&&** and **||** examines only the first element of the operands resulting in a single length logical vector.

Zero is considered **FALSE** and non-zero numbers are taken as **TRUE**. Let's see an example for this:

```
x <- c(TRUE, FALSE, 0, 6)
y <- c(FALSE, TRUE, FALSE, TRUE)
!x
x & y
x && y
x | y
x || y
```

Output

```
[1] FALSE TRUE TRUE FALSE
[1] FALSE FALSE FALSE TRUE
[1] FALSE
[1] TRUE TRUE FALSE TRUE
[1] TRUE
```

Assignment Operators:

Operator	Description	Example
=, <-, <<-	Left word Assigns	x <- 5, x=10, x<<-20

Operator	Description	Example
<code>->, ->></code>	Right word Assigns	<code>5 -> x, 20->>y</code>

Miscellaneous operators are used to manipulate data:

Operator	Description	Example
<code>:</code>	<i>Creates a series of numbers in a sequence</i>	<code>x <- 1:10</code>
<code>%in%</code>	<i>Find out if an element belongs to a vector</i>	<code>x %in% y</code>
<code>%*%</code>	<i>Matrix Multiplication</i>	<code>x <- Matrix1 %*% Matrix2</code>

Operation on Vectors

The above mentioned operators work on [vectors](#). The variables used above were in fact single element vectors.

We can use the function `c()` (as in concatenate) to make vectors in R.

All operations are carried out in element-wise fashion. Here is an example.

```
x <- c(2, 8, 3)
y <- c(6, 4, 1)
x + y
x > y
```

Output

```
[1] 8 12 4
[1] FALSE TRUE TRUE
```

When there is a mismatch in length (number of elements) of operand vectors, the elements in the shorter one are recycled in a cyclic manner to match the length of the longer one.

R will issue a warning if the length of the longer vector is not an integral multiple of the shorter vector.

```
x <- c(2, 1, 8, 3)
y <- c(9, 4)
```

```
x + y
x - 1
x + c(1, 2, 3)
```

Output

```
[1] 11 5 17 7
[1] 1 0 7 2
[1] 3 3 11 4
Warning message:
In x + c(1, 2, 3) :
  longer object length is not a multiple of shorter object length
```

VIVA QUESTIONS AND ANSWERS

Q1: What is R programming language? A1: R is a programming language and environment specifically designed for statistical computing and graphics. It provides a wide range of statistical and graphical techniques and is widely used by statisticians, data analysts, and researchers.

Q2: How do you assign values to variables in R? A2: In R, you can assign values to variables using the assignment operator <=.

For example:

```
x <- 10
```

```
name <- "John"
```

```
is_true <- TRUE
```

Q3: What are vectors in R? A3: Vectors are one-dimensional arrays that can hold elements of the same data type. They are created using the `c()` function. For example:

```
numbers <- c(1, 2, 3, 4, 5)
```

```
names <- c("John", "Doe", "Alice")
```

Q4: How do you create a data frame in R? A4: Data frames are used to store tabular data in R. They can hold different types of data in different columns. You can create a data frame using the `data.frame()` function. For example:

```
student_data <- data.frame(
```

```
  name = c("John", "Doe", "Alice"),
```

```
  age = c(25, 30, 28),
```

```
  grade = c("A", "B", "B+") )
```

Q5: What is a function in R? A5: In R, functions are blocks of code that perform a specific task. They are defined using the `function()` keyword. You can create your own functions or use built-in functions. For example:

```
square <- function(x) {  
  return(x^2)  
}
```

Q6: How do you write conditional statements in R?

A6: R supports conditional statements like if-else statements. You can use them to execute different blocks of code based on certain conditions. For example:

```
x <- 10  
  
if (x > 5) {  
  print("x is greater than 5")  
} else {  
  print("x is less than or equal to 5")  
}
```

7. What is a loop, and how do you use it in R?

Ana: A loop is used to repeatedly execute a block of code. R supports various types of loops, including `for` and `while` loops. For example:

```
for (i in 1:5) {  
  print(i)  
}
```

8. How can you check the type of a variable in R?

Ans: You can check the type of a variable in R using the `class()` function. For example:

```
class(x) # Output: "numeric"
```

9. What control structures are available in R?

Answer: R supports various control structures such as if-else statements, for loops, while loops, and switch statements. These structures allow you to control the flow of execution in your code.

Program 3: Write R command to

- i. Illustrate summation, subtraction, multiplication, and division operations on vectors using vectors.
- ii. Enumerate multiplication and division operations between matrices and vectors in R console

DESCRIPTION:

Vector Arithmetic Operations

We can perform arithmetic operations on vectors, like addition, subtraction, multiplication and division. If the vectors are not of same length, then [Vector Recycling](#) happens implicitly.

Addition

Addition operator takes two vectors as operands, and returns the result of sum of two vectors.

$a + b$

Example

In the following program, we create two [integer vectors](#) and add them using Addition Operator.

Example.R

```
a <- c(10, 20, 30, 40, 50)
b <- c(1, 3, 5, 7, 9)
result <- a + b
print(result)
```

Output

```
[1] 11 23 35 47 59
```

ADVERTISEMENT

Subtraction

Subtraction operator takes two vectors as operands, and returns the result of difference of two vectors.

$a - b$

Example

In the following program, we create two [integer vectors](#) and find their difference using Subtraction Operator.

Example.R

```
a <- c(10, 20, 30, 40, 50)
b <- c(1, 3, 5, 7, 9)
result <- a - b
print(result)
```

Output

```
[1]  9 17 25 33 41
```

Multiplication

Multiplication operator takes two vectors as operands, and returns the result of product of two vectors.

`a * b`

Example

In the following program, we create two [integer vectors](#) and find their product using Multiplication Operator.

Example.R

```
a <- c(10, 20, 30, 40, 50)
b <- c(1, 3, 5, 7, 9)
result <- a * b
print(result)
```

Output

```
[1] 10 60 150 280 450
```

Division

Division operator takes two vectors as operands, and returns the result of division of two vectors.

`a / b`

Example

In the following program, we create two [integer vectors](#) and divide them using Division Operator.

Example.R

```
a <- c(10, 20, 30, 40, 50)
b <- c(1, 3, 5, 7, 9)
result <- a / b
print(result)
```

Output

```
[1] 10.000000 6.666667 6.000000 5.714286 5.555556
```

ii. Enumerate multiplication and division operations between matrices and vectors in R console

In R, you can perform multiplication and division operations between matrices and vectors using various functions and operators. Here are some examples:

Multiplication Operations:

Matrix Multiplication (%*% operator):

- Multiply a matrix by another matrix.

```
A <- matrix(1:4, nrow = 2)
B <- matrix(1:6, nrow = 2)
C <- A %*% B
print(C)
```

Matrix-Vector Multiplication (%*% operator):

- Multiply a matrix by a vector.

```
A <- matrix(1:4, nrow = 2)
v <- c(1, 2)
result <- A %*% v
print(result)
```

Element-Wise Multiplication (* operator):

- Multiply corresponding elements of two matrices or vectors.

```
A <- matrix(1:4, nrow = 2)
B <- matrix(1:4, nrow = 2)
C <- A * B
print(C)
```

Division Operations:

1. Matrix Division:

- There isn't a division operator for matrices in the same way as multiplication.
- Instead, you typically use matrix inversion.

```
A <- matrix(c(1, 2, 3, 4), nrow = 2)
A_inv <- solve(A)
```

```
print(A_inv)
```

Element-Wise Division (/ operator):

- Divide corresponding elements of two matrices or vectors.

```
A <- matrix(1:4, nrow = 2)
B <- matrix(1:4, nrow = 2)
C <- A / B
print(C)
```

VIVA QUESTIONS AND ANSWERS

1.What is the result of adding two vectors in R?

Ans: The result is a new vector where each element is the sum of the corresponding elements of the two input vectors.

2.How does R handle element-wise multiplication between vectors?

Ans: R multiplies corresponding elements of the vectors together to produce a new vector of the same length.

3.What is the difference between matrix multiplication and element-wise multiplication in R?

Ans: Matrix multiplication involves the dot product of rows and columns according to matrix algebra rules, while element-wise multiplication simply multiplies corresponding elements of matrices or vectors.

4.How can you perform matrix division in R?

Ans: In R, you typically perform matrix division by computing the inverse of the matrix and then multiplying it by the vector you want to divide.

5.Why is matrix division more computationally expensive than matrix multiplication in R?

Ans: Matrix division involves matrix inversion, which is computationally intensive and may not always be feasible for large matrices or matrices with certain properties.

Program 4: Write Rcommand to

- i) Illustrates the usage of Vector subsetting and Matrix subsetting
- ii) Write a program to create an array of 3×3 matrixes with 3 rows and 3 columns.

AIM: Illustration of vector & matrix subsetting. Create a matrix of 33 using arrays

DESCRIPTION:

Vector Subsetting

Vector subsetting in R involves selecting specific elements or subsets from a vector based on certain conditions or indices. Here's how you can perform vector subsetting in R with examples:

Example 1: Basic Vector Subsetting

Create a vector

```
my_vector <- c(10, 20, 30, 40, 50)
```

Select elements at specific indices

```
subset_vector <- my_vector[c(2, 4)] # Selecting elements at index 2 and 4
```

```
print(subset_vector) # Output: [1] 20 40
```

In this example, we create a vector `my_vector` with elements 10, 20, 30, 40, and 50. Then, we use vector subsetting to extract elements at indices 2 and 4, which are 20 and 40, respectively.

Example 2: Conditional Vector Subsetting

Create a vector

```
my_vector <- c(10, 20, 30, 40, 50)
```

Select elements based on a condition

```
subset_vector <- my_vector[my_vector > 30] # Select elements greater than 30
```

```
print(subset_vector) # Output: [1] 40 50
```

In this example, we create a vector `my_vector`. We use vector subsetting with a condition (`my_vector > 30`) to extract elements greater than 30.

Example 3: Named Vector Subsetting

Create a named vector

```
my_vector <- c(apple = 10, banana = 20, cherry = 30, durian = 40, elderberry = 50)
```

```
# Select elements by name
```

```
subset_vector <- my_vector[c("banana", "durian")] # Select elements by name
```

```
print(subset_vector) # Output: banana durian
```

```
      #      20  40
```

In this example, we create a named vector `my_vector`. We use vector subsetting with names to select elements by their names, in this case, "banana" and "durian".

Matrix Subsetting

Matrix subsetting in R involves selecting specific elements, rows, or columns from a matrix based on certain conditions or indices. Here's how you can perform matrix subsetting in R with examples:

Example 1: Basic Matrix Subsetting

```
# Create a matrix
```

```
my_matrix <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
```

```
# Print the original matrix
```

```
print(my_matrix)
```

```
# Output:
```

```
#   [,1] [,2] [,3] [,4]
```

```
# [1,]  1  2  3  4
```

```
# [2,]  5  6  7  8
```

```
# [3,]  9 10 11 12
```

```
# Select specific elements from the matrix
```

```
subset_matrix <- my_matrix[2, 3] # Select element at row 2, column 3
```

```
print(subset_matrix) # Output: [1] 7
```

In this example, we create a 3x4 matrix `my_matrix` with values from 1 to 12. Then, we use matrix subsetting to extract the element at the 2nd row and 3rd column, which is 7.

Example 2: Row and Column Subsetting

```
# Create a matrix
```

```
my_matrix <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
```

```
# Print the original matrix
```

```
print(my_matrix)
```

```
# Output:
```

```
#      [,1] [,2] [,3] [,4]
```

```
# [1,]  1  2  3  4
```

```
# [2,]  5  6  7  8
```

```
# [3,]  9 10 11 12
```

```
# Select specific rows and columns from the matrix
```

```
subset_matrix <- my_matrix[2:3, 2:3] # Select rows 2 and 3, columns 2 and 3
```

```
print(subset_matrix)
```

```
# Output:
```

```
#      [,1] [,2]
```

```
# [1,]  6  7
```

```
# [2,] 10 11
```

In this example, we use matrix subsetting to extract a subset of the matrix containing rows 2 and 3, and columns 2 and 3.

Matrix subsetting allows you to extract specific elements, rows, or columns from a matrix based on various conditions or indices.

ii. Write a program to create an array of 3x3 matrixes with 3 rows and 3 columns.

Below is a program in R that creates an array consisting of 3 3x3 matrices, each with 3 rows and 3 columns:

In this program:

- We create three 3x3 matrices (**matrix1**, **matrix2**, **matrix3**) using the **matrix()** function with the **nrow** and **ncol** parameters set to 3.
- Then, we combine these matrices into an array using the **array()** function. We pass the matrices as a vector and specify the dimensions of the array as **dim = c(3, 3, 3)** indicating 3 matrices each of size 3x3.
- Finally, we print the resulting array of matrices using the **print()** function.

You can modify the content of **matrix1**, **matrix2**, and **matrix3** to contain any values you want.

SOURCE CODE:

```
# Create three 3x3 matrices
```

```
matrix1 <- matrix(1:9, nrow = 3, ncol = 3)
```

```
matrix2 <- matrix(10:18, nrow = 3, ncol = 3)
```

```
matrix3 <- matrix(19:27, nrow = 3, ncol = 3)

# Combine the matrices into an array

array_of_matrices <- array(c(matrix1, matrix2, matrix3), dim = c(3, 3, 3))

# Print the array of matrices

print(array_of_matrices)
```

VIVA QUESTIONS & ANSWERS

Vectors:

Q1: What is a vector in R?

A1: In R, a vector is a one-dimensional array that can store elements of the same data type. It can be created using the `c()` function.

Q2: How can you access elements in a vector?

A2: Elements in a vector can be accessed using indices. For example, `my_vector[3]` will give you the third element of the vector `my_vector`.

Q3: Illustrate the usage of vector subsetting in R.

A3:

Example of vector subsetting

```
my_vector <- c(10, 20, 30, 40, 50)

subset_vector <- my_vector[c(2, 4)] # Select elements at index 2 and 4

print(subset_vector)
```

Q4: Explain the difference between named and unnamed vectors.

A4: Named vectors have labels assigned to each element, making it possible to access elements using names. Unnamed vectors do not have such labels.

Matrices:

Q5: What is a matrix in R?

A5: A matrix in R is a two-dimensional data structure with rows and columns, where each element must be of the same data type.

Q6: How can you create a matrix in R?

A6: You can create a matrix using the `matrix()` function. For example, `matrix(1:6, nrow = 2, ncol = 3)` creates a 2x3 matrix with values from 1 to 6.

Q7: Explain the process of matrix subsetting.

A7: Matrix subsetting involves selecting specific elements, rows, or columns from a matrix. You can use indices or conditions to extract the desired subset.

Q8: Provide an example of creating and subsetting a matrix in R.

A8:

Example of creating and subsetting a matrix

```
my_matrix <- matrix(1:9, nrow = 3, ncol = 3)
```

```
subset_matrix <- my_matrix[2:3, 2:3] # Select rows 2 and 3, columns 2 and 3
```

```
print(subset_matrix)
```

9. How can you find the dimensions of a matrix in R?

A9: You can use the `dim()` function in R to find the dimensions of a matrix. For example: `matrix_dimensions <- dim(my_matrix)`.

10. Explain the difference between a vector and a matrix in R.

A10: A vector in R is a one-dimensional array, whereas a matrix in R is a two-dimensional array. Vectors have only one dimension (rows or columns), while matrices have both rows and columns.

Program 5: Write an R program to draw i) Pie chart ii) 3D Pie Chart, iii) Bar Chart along with chart legend by considering suitable CSV file

AIM:

DESCRIPTION:

To create pie charts, 3D pie charts, and bar charts in R using a CSV file, you can use the `read.csv()` function to read the data from the CSV file and then use the relevant plotting functions from the `ggplot2` package. Make sure to install the package if you haven't already by running `install.packages("ggplot2")`. Below is an example R program:

SOURCE CODE:

Install and load necessary packages

```
if (!requireNamespace("ggplot2", quietly = TRUE)) {  
  install.packages("ggplot2")  
}
```

```
library(ggplot2)
```

Read data from CSV file

```
data <- read.csv("your_file.csv") # Replace "your_file.csv" with your actual file name
```

i) Pie chart

```
pie_chart <- ggplot(data, aes(x = "", y = YourColumnName)) +  
  geom_bar(stat = "identity", fill = "blue") +  
  coord_polar("y", start = 0) +  
  ggtitle("Pie Chart") +  
  theme_minimal()
```

ii) 3D Pie Chart

```
pie3D_chart <- ggplot(data, aes(x = "", y = YourColumnName, fill = YourColumnName)) +  
  geom_bar(stat = "identity", color = "white") +  
  coord_polar("y", start = 0) +  
  ggtitle("3D Pie Chart") +  
  theme_minimal()
```

iii) Bar Chart with legend

```
bar_chart <- ggplot(data, aes(x = XAxisColumnName, y = YourColumnName, fill = YourColumnName)) +  
  geom_bar(stat = "identity", position = "dodge", color = "white") +  
  ggtitle("Bar Chart") +  
  labs(x = "X-axis Label", y = "Y-axis Label") +  
  theme_minimal() +  
  theme(legend.position = "top")
```

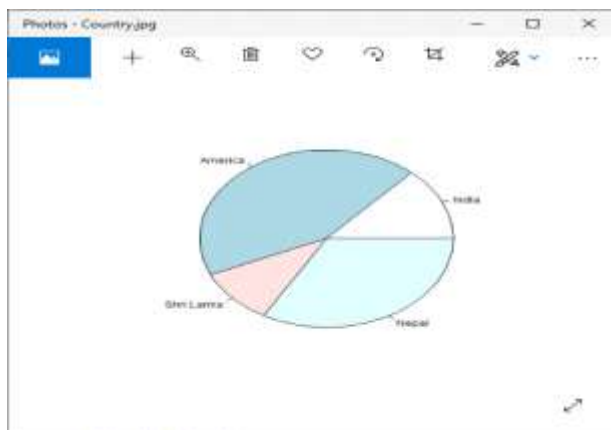
Plot the charts

```
print(pie_chart)  
print(pie3D_chart)  
print(bar_chart)
```

Example

```
# Creating data for the graph.  
x <- c(20, 65, 15, 50)  
labels <- c("India", "America", "Shri Lanka", "Nepal")  
# Giving the chart file a name.  
png(file = "Country.jpg")  
# Plotting the chart.  
pie(x, labels)  
# Saving the file.  
dev.off()
```

Output:



Title and color

A pie chart has several more features that we can use by adding more parameters to the `pie()` function. We can give a title to our pie chart by passing the `main` parameter. It tells the title of the pie chart to the `pie()` function. Apart from this, we can use a rainbow colour pallet while drawing the chart by passing the `col` parameter.

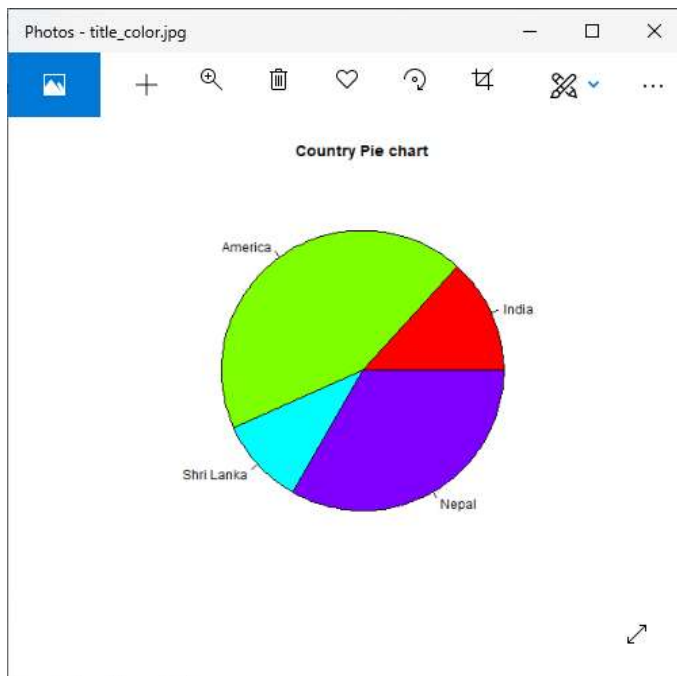
Note: The length of the pallet will be the same as the number of values that we have for the chart. So for that, we will use `length()` function.

Let's see an example to understand how these methods work in creating an attractive pie chart with title and color.

Example

1. # Creating data for the graph.
2. `x <- c(20, 65, 15, 50)`
3. `labels <- c("India", "America", "Shri Lanka", "Nepal")`
4. # Giving the chart file a name.
5. `png(file = "title_color.jpg")`
6. # Plotting the chart.
7. `pie(x, labels, main = "Country Pie chart", col = rainbow(length(x)))`
8. # Saving the file.
9. `dev.off()`

Output:



Example

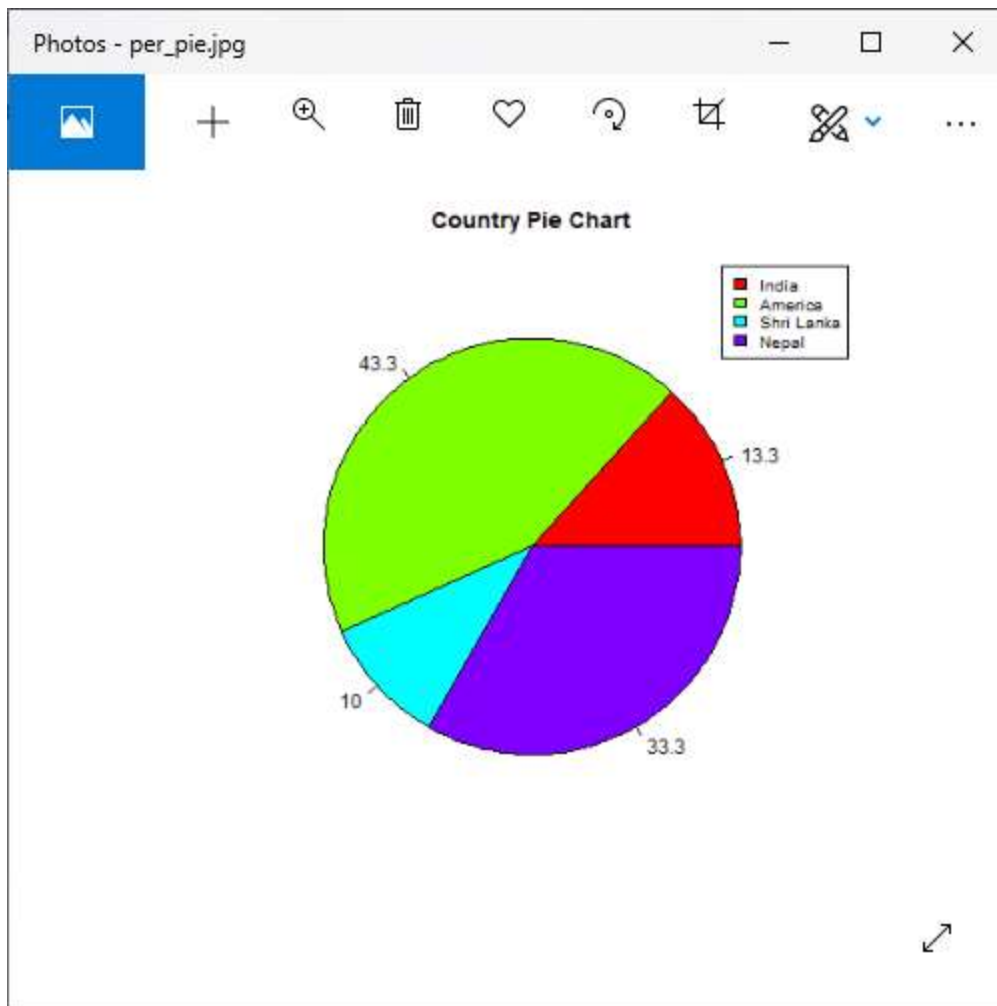
1. # Creating data for the graph.
2. `x <- c(20, 65, 15, 50)`
3. `labels <- c("India", "America", "Shri Lanka", "Nepal")`

```

4. pie_percent<- round(100*x/sum(x), 1)
5. # Giving the chart file a name.
6. png(file = "per_pie.jpg")
7. # Plotting the chart.
8. pie(x, labels = pie_percent, main = "Country Pie Chart",col = rainbow(length(x)))
9. legend("topright", c("India", "America", "Shri Lanka", "Nepal"), cex = 0.8,
10. fill = rainbow(length(x)))
11. #Saving the file.
12. dev.off()

```

Output:



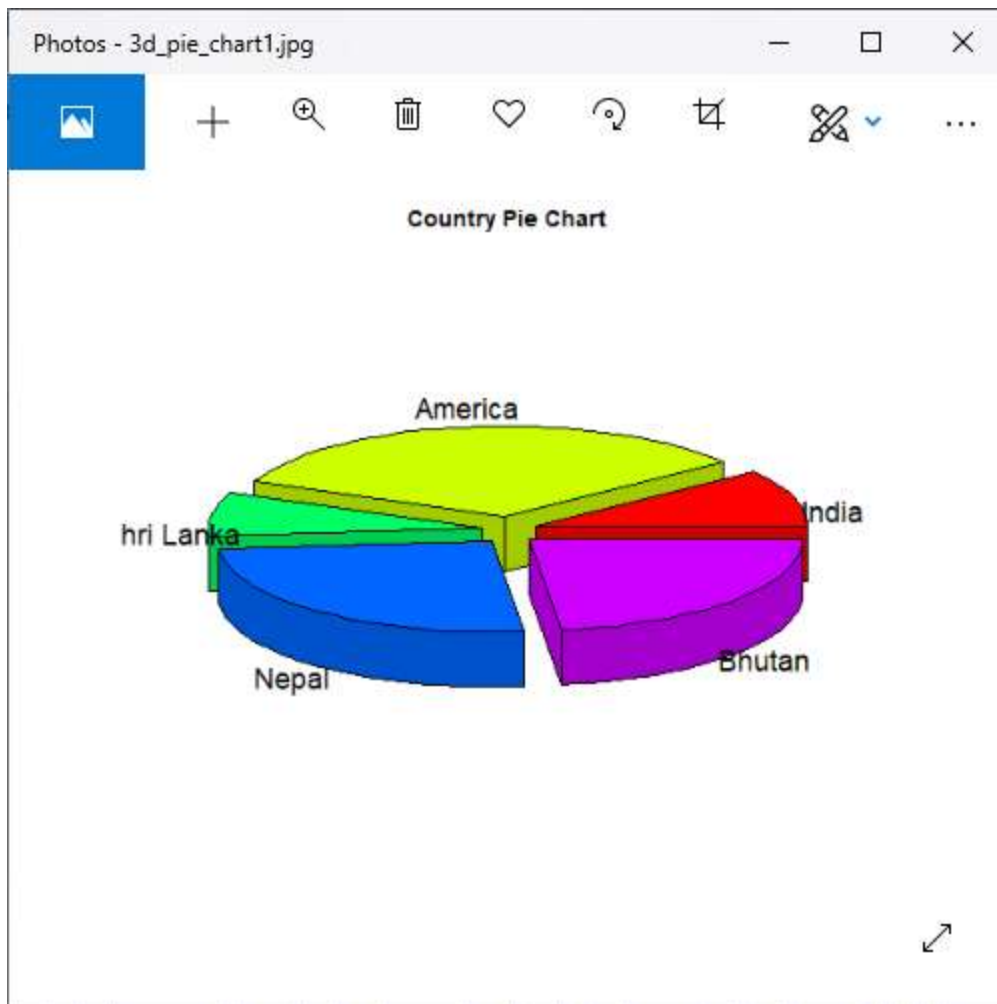
3 Dimensional Pie Chart

In R, we can also create a three-dimensional pie chart. For this purpose, R provides a plotrix package whose `pie3D()` function is used to create an attractive 3D pie chart. The parameters of `pie3D()` function remain same as `pie()` function. Let's see an example to understand how a 3D pie chart is created with the help of this function.

Example

1. # Getting the library.
2. library(plotrix)
3. # Creating data for the graph.
4. x <- c(20, 65, 15, 50,45)
5. labels <- c("India", "America", "Shri Lanka", "Nepal","Bhutan")
6. # Give the chart file a name.
7. png(file = "3d_pie_chart1.jpg")
8. # Plot the chart.
9. pie3D(x,labelslabels = labels,explode = 0.1, main = "Country Pie Chart")
10. # Save the file.
11. dev.off()

Output:

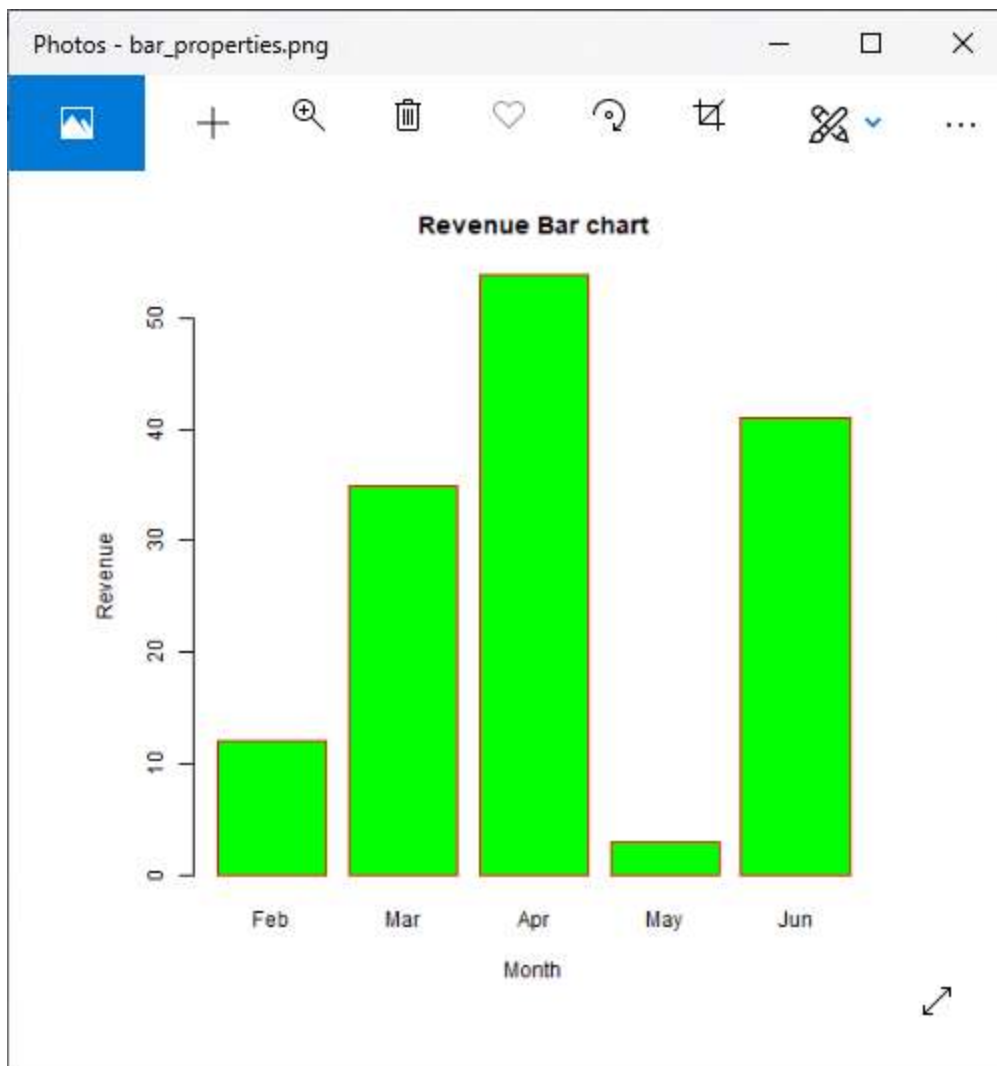


Bar Plots:

Example

1. # Creating the data for Bar chart
2. `H <- c(12,35,54,3,41)`
3. `M<- c("Feb","Mar","Apr","May","Jun")`
- 4.
5. # Giving the chart file a name
6. `png(file = "bar_properties.png")`
- 7.
8. # Plotting the bar chart
9. `barplot(H,names.arg=M,xlab="Month",ylab="Revenue",col="Green",`
10. `main="Revenue Bar chart",border="red")`
11. # Saving the file
12. `dev.off()`

Output:



Group Bar Chart & Stacked Bar Chart

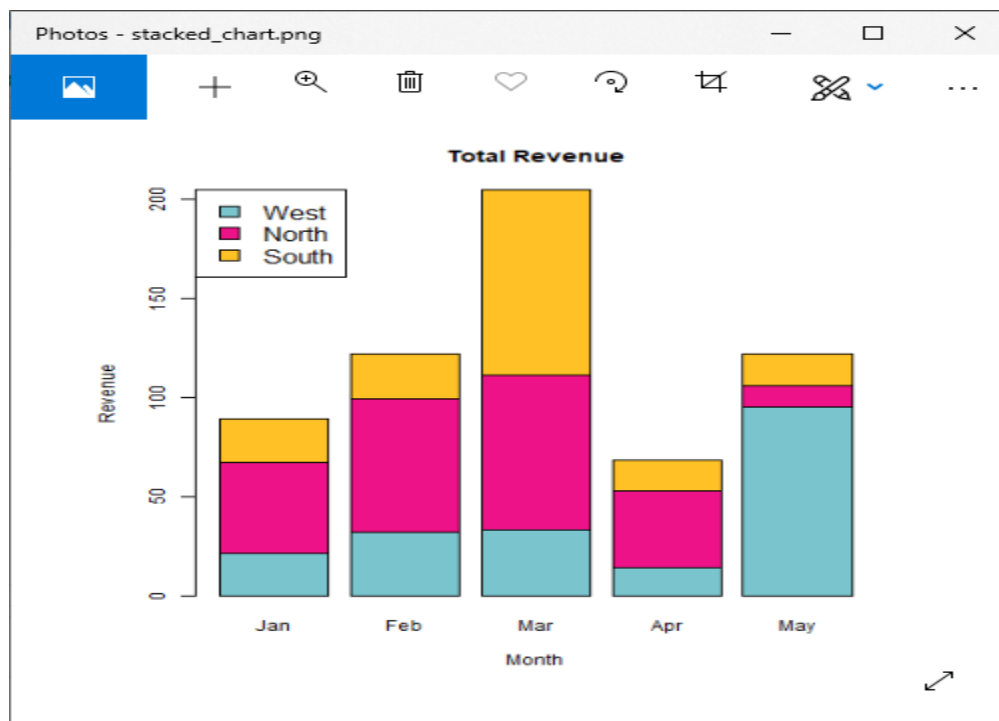
We can create bar charts with groups of bars and stacks using matrices as input values in each bar. One or more variables are represented as a matrix that is used to construct group bar charts and stacked bar charts.

Let's see an example to understand how these charts are created.

Example

1. `library(RColorBrewer)`
2. `months <- c("Jan","Feb","Mar","Apr","May")`
3. `regions <- c("West","North","South")`
4. `# Creating the matrix of the values.`
5. `Values <- matrix(c(21,32,33,14,95,46,67,78,39,11,22,23,94,15,16), nrow = 3, ncol = 5, byrow = TRUE)`
6. `# Giving the chart file a name`
7. `png(file = "stacked_chart.png")`
8. `# Creating the bar chart`
9. `barplot(Values, main = "Total Revenue", names.arg = months, xlab = "Month", ylab = "Revenue", ccol = c("cadetblue3","deep pink2","goldenrod1"))`
10. `# Adding the legend to the chart`
11. `legend("topleft", regions, cex = 1.3, fill = c("cadetblue3","deeppink2","goldenrod1"))`
- 12.
13. `# Saving the file`
14. `dev.off()`

Output:



VIVA QUESTIONS & ANSWERS

1. **Question:** What is a pie chart, and when is it appropriate to use one?

Answer: A pie chart is a circular statistical graphic that is divided into slices to illustrate numerical proportions. It is best used when you want to represent the relative proportions of different categories within a dataset. Pie charts are effective for displaying simple proportions and highlighting the composition of a whole.

2. **Question:** How can you create a pie chart in R?

Answer: In R, you can create a pie chart using the `pie()` function. Here's a simple example:

Create a vector with data

```
data <- c(10, 20, 30, 40)
```

Create a pie chart

```
pie(data, labels = c("A", "B", "C", "D"))
```

3. **Question:** What is a 3D pie chart, and how does it differ from a regular pie chart?

Answer: A 3D pie chart is similar to a regular pie chart but adds a three-dimensional effect to the chart. It typically includes depth to the slices, making it appear as if the slices are emerging from the center. However, 3D pie charts can sometimes distort the proportions of the data and make it difficult to accurately compare slice sizes.

4.Question: How can you create a 3D pie chart in R?

Answer: In R, you can create a 3D pie chart using the `plotrix` package. Here's a simple example:

Install and load the plotrix package

```
install.packages("plotrix")
```

```
library(plotrix)
```

Create a vector with data

```
data <- c(10, 20, 30, 40)
```

Create a 3D pie chart

```
pie3D(data, labels = c("A", "B", "C", "D"), explode = 0.1)
```

5. **Question:** When would you choose to use a bar plot instead of a pie chart?

Answer: Bar plots are generally used to compare different categories or groups against one another. They are more suitable when you want to show the magnitude of each category or group individually and compare them easily. Bar plots are especially useful when dealing with larger datasets or when precise comparisons between categories are necessary.

6.Question: How can you create a bar plot in R?

Answer: In R, you can create a bar plot using the `ggplot2` package. Here's a simple example:

Install and load the ggplot2 package

```
install.packages("ggplot2")
```

```
library(ggplot2)
```

```
# Create a sample data frame
```

```
data <- data.frame(
```

```
  Category = c("A", "B", "C", "D"),
```

```
  Value = c(10, 20, 30, 40)
```

```
)
```

```
# Create a bar plot
```

```
ggplot(data, aes(x = Category, y = Value, fill = Category)) +
```

```
  geom_bar(stat = "identity") +
```

```
  labs(title = "Bar Plot", x = "Categories", y = "Values")
```

Program 6: Create a CSV file having Speed and Distance attributes with 1000 records. Write a program to draw

- i) **Box plots**
- ii) **Histogram**
- iii) **Line Graph**
- iv) **Multiple line graphs**
- v) **Scatterplot**

To demonstrate the relation between the cars speed and the distance.

AIM:

DESCRIPTION:

Box Plot Example

Giving a name to the chart file.

```
png(file = "boxplot.png")
```

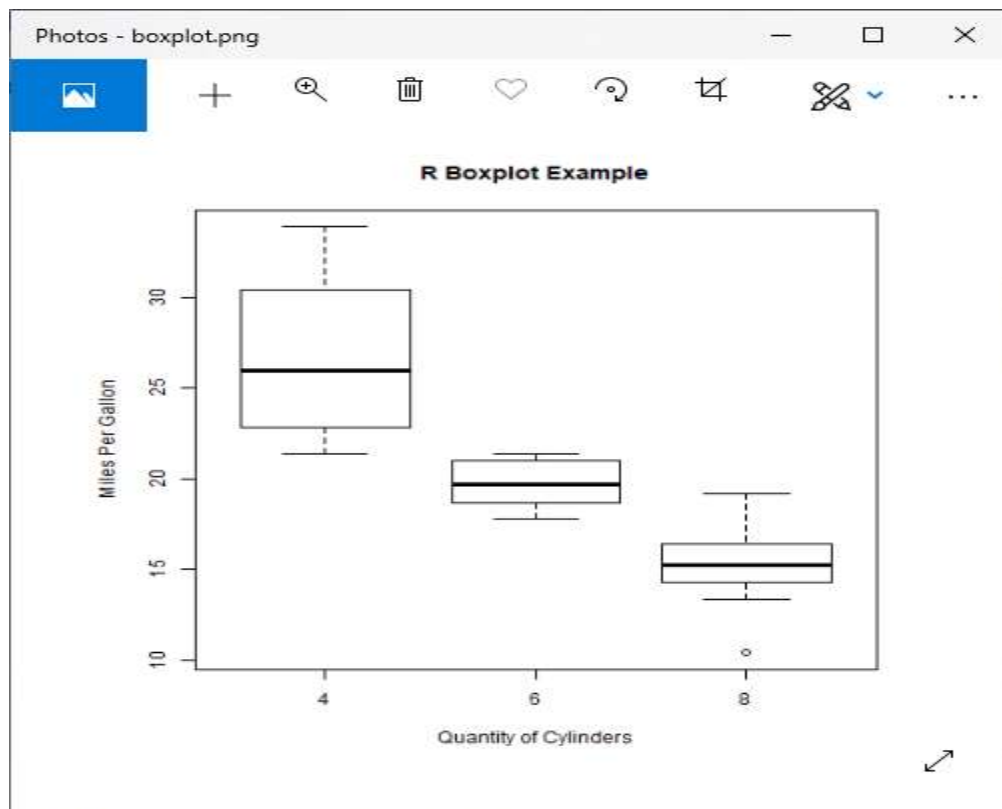
Plotting the chart.

```
boxplot(mpg ~ cyl, data = mtcars, xlab = "Quantity of Cylinders",  
        ylab = "Miles Per Gallon", main = "R Boxplot Example")
```

Save the file.

```
dev.off()
```

Output:



Histogram Example

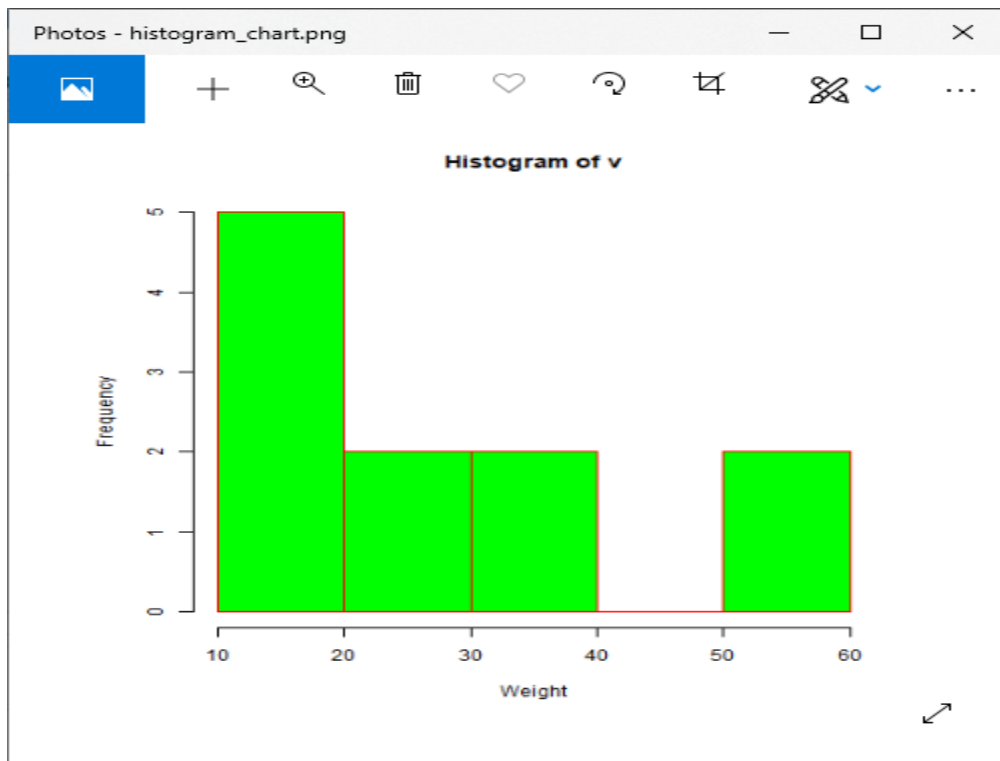
Creating data for the graph.

```
v <- c(12,24,16,38,21,13,55,17,39,10,60)
```

```
# Giving a name to the chart file.
png(file = "histogram_chart.png")

# Creating the histogram.
hist(v,xlab = "Weight",ylab="Frequency",col = "green",border = "red")
# Saving the file.
dev.off()
```

Output:



Let's see some more examples in which we have used different parameters of hist() function to add more functionality or to create a more attractive chart.

Example: Use of xlim & ylim parameter

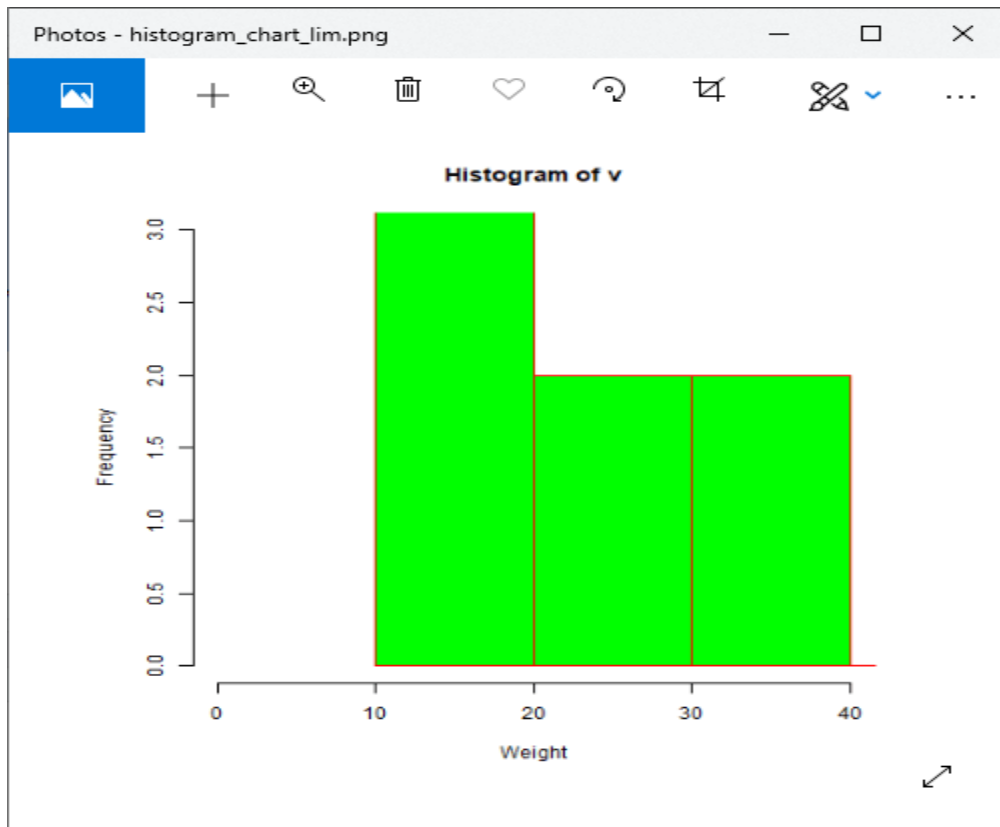
```
# Creating data for the graph.
v <- c(12,24,16,38,21,13,55,17,39,10,60)

# Giving a name to the chart file.
png(file = "histogram_chart_lim.png")

# Creating the histogram.
hist(v,xlab = "Weight",ylab="Frequency",col = "green",border = "red",xlim = c(0,40), ylim = c(0,3), breaks = 5)

# Saving the file.
dev.off()
```

Output:



Example: Finding return value of hist()

1. # Creating data for the graph.
2. `v <- c(12,24,16,38,21,13,55,17,39,10,60)`
- 3.
4. # Giving a name to the chart file.
5. `png(file = "histogram_chart_lim.png")`
6. # Creating the histogram.
7. `m<-hist(v)`
8. `m`

Output:

```

C:\Users\ajeet>cd R
C:\Users\ajeet\R>Rscript histo.R
$breaks
[1] 10 20 30 40 50 60

$counts
[1] 5 2 2 0 2

$density
[1] 0.04545455 0.01818182 0.01818182 0.00000000 0.01818182

$mids
[1] 15 25 35 45 55

$xname
[1] "v"

$equidist
[1] TRUE

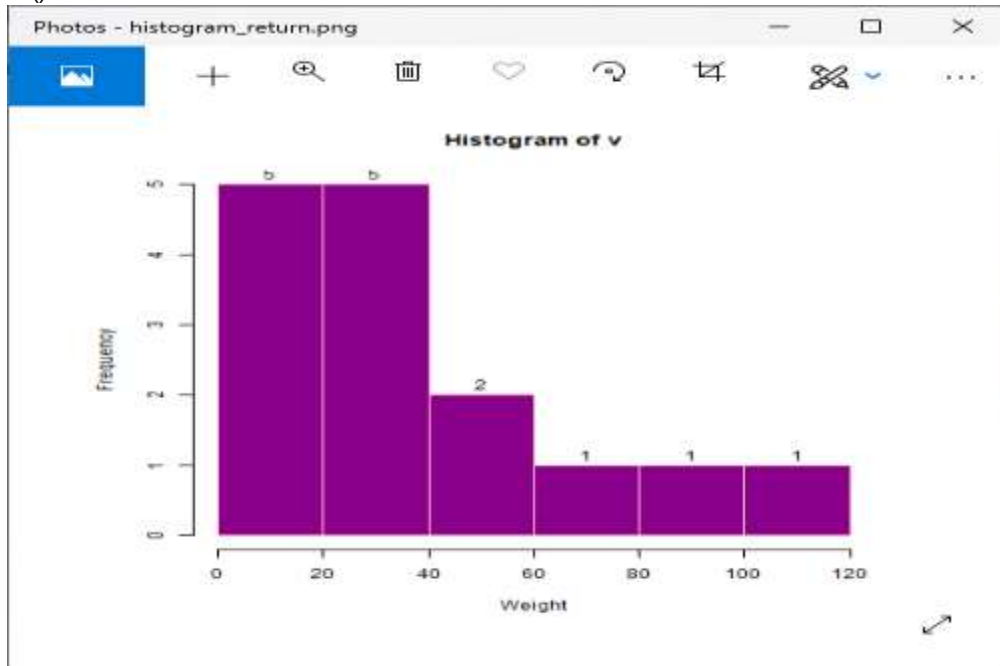
attr(,"class")
[1] "histogram"

C:\Users\ajeet\R>_

```

Example: Using histogram return values for labels using text()

```
# Creating data for the graph.
v <- c(12,24,16,38,21,13,55,17,39,10,60,120,40,70,90)
# Giving a name to the chart file.
png(file = "histogram_return.png")
# Creating the histogram.
m<-hist(v,xlab = "Weight",ylab="Frequency",col = "darkmagenta",border = "pink", breaks = 5)
#Setting labels
text(m$mids,m$counts,labels=m$counts, adj=c(0.5, -0.5))
# Saving the file.
dev.off()
```



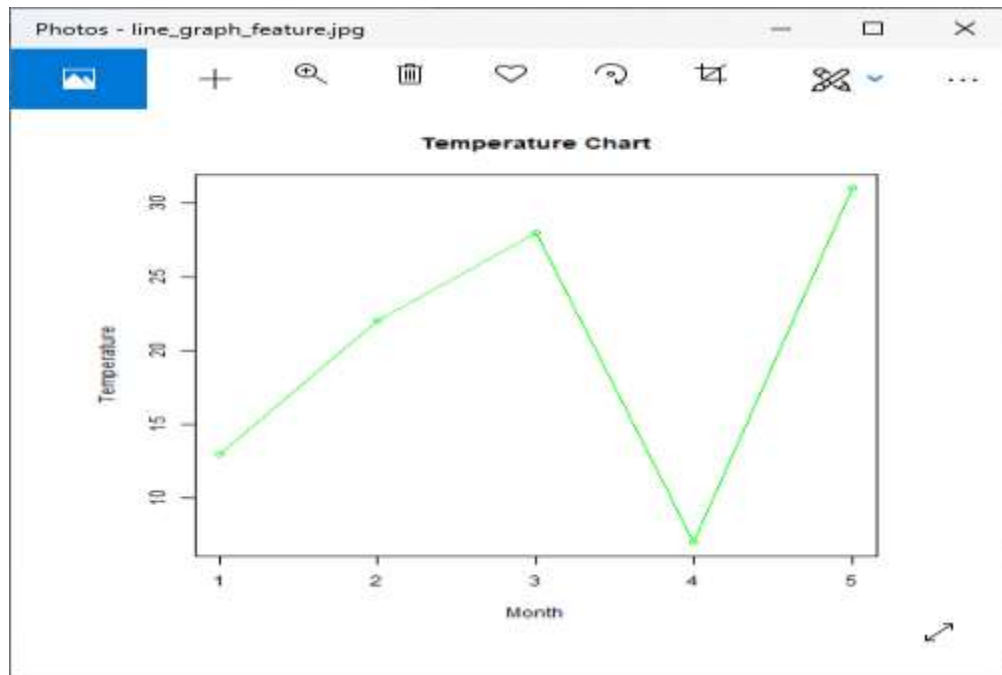
Line Chart Title, Color, and Labels

Like other graphs and charts, in line chart, we can add more features by adding more parameters. We can add the colors to the lines and points, add labels to the axis, and can give a title to the chart. Let's see an example to understand how these parameters are used in plot() function to create an attractive line graph.

Example

```
# Creating the data for the chart.
v <- c(13,22,28,7,31)
# Giving a name to the chart file.
png(file = "line_graph_feature.jpg")
# Plotting the bar chart.
plot(v,type = "o",col="green",xlab="Month",ylab="Temperature")
# Saving the file.
dev.off()
```

Output:



Line Charts Containing Multiple Lines

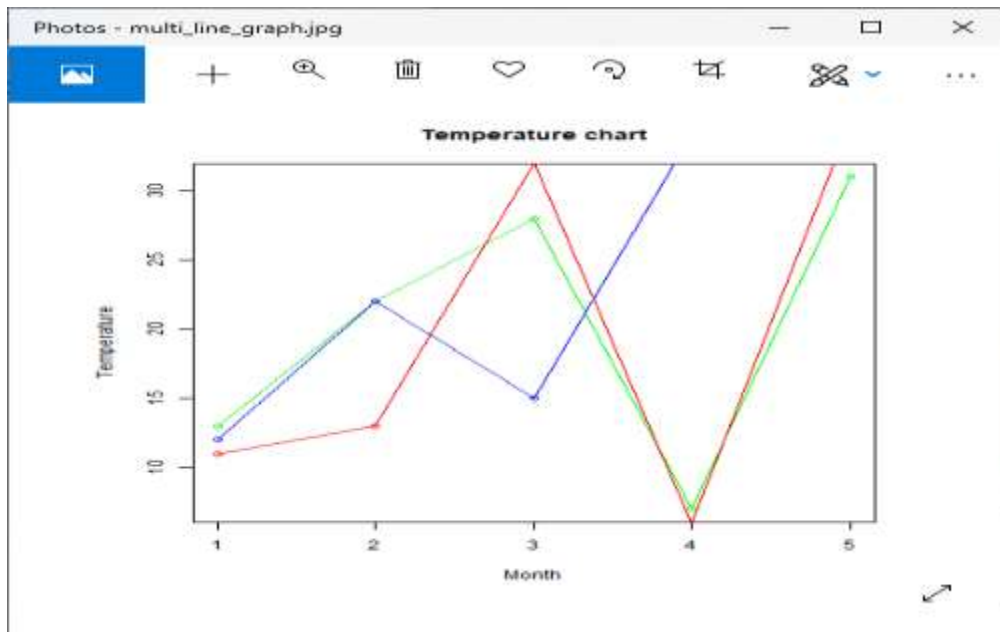
In our previous examples, we created line graphs containing only one line in each graph. R allows us to create a line graph containing multiple lines. R provides `lines()` function to create a line in the line graph.

The `lines()` function takes an additional input vector for creating a line. Let's see an example to understand how this function is used:

Example

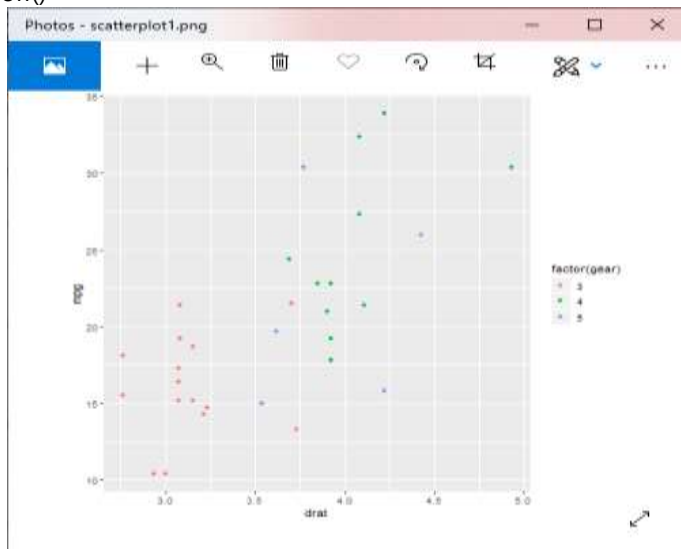
```
# Creating the data for the chart.
v <- c(13,22,28,7,31)
w <- c(11,13,32,6,35)
x <- c(12,22,15,34,35)
# Giving a name to the chart file.
png(file = "multi_line_graph.jpg")
# Plotting the bar chart.
plot(v,type = "o",col="green",xlab="Month",ylab="Temperature")
lines(w, type = "o", col = "red")
lines(x, type = "o", col = "blue")
# Saving the file.
dev.off()
```

Output:



Example 1: Scatterplot with groups

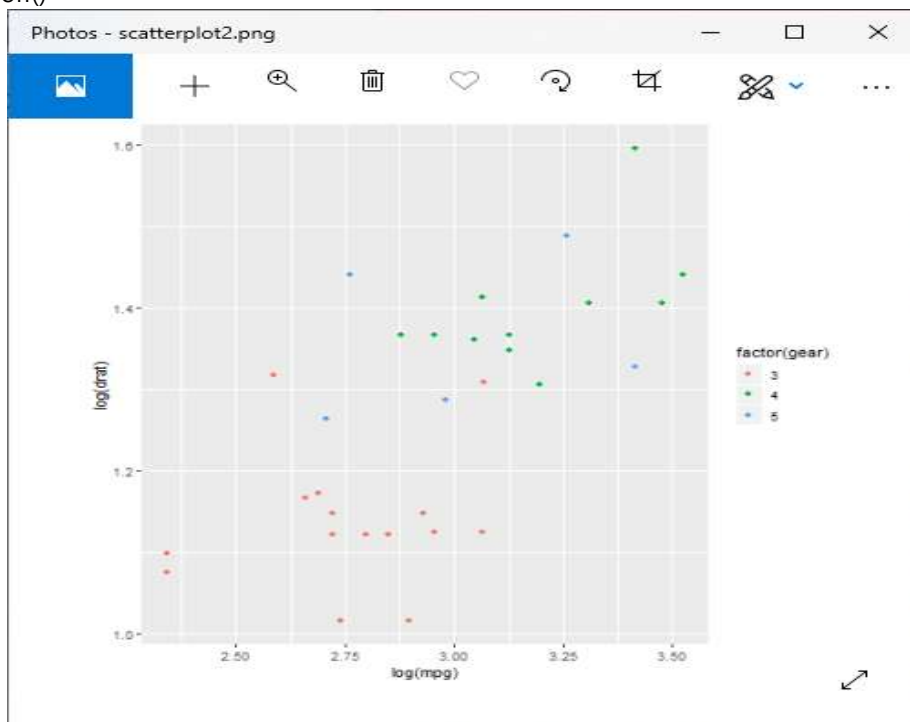
```
#Loading ggplot2 package
library(ggplot2)
# Giving a name to the chart file.
png(file = "scatterplot1.png")
# Plotting the chart using ggplot() and geom_point() functions.
#The aes() function inside the geom_point() function controls the color of the group.
ggplot(mtcars, aes(x = drat, y = mpg)) +
  geom_point(aes(color=factor(gear)))
# Saving the file.
dev.off()
```



Example 2: Changes in axis

```
#Loading ggplot2 package
library(ggplot2)
# Giving a name to the chart file.
png(file = "scatterplot2.png")
# Plotting the chart using ggplot() and geom_point() functions.
```

```
#The aes() function inside the geom_point() function controls the color of the group.
ggplot(mtcars, aes(x = log(mpg), y = log(drat))) + geom_point(aes(color=factor(gear)))
# Saving the file.
dev.off()
```



VIVA VOICE QUESTIONS & ANSWERS

1. Bar Plot:

- *Question:* "Can you describe the trend in the bar plot representing sales data over the past year?"
- *Answer:* "Certainly! The bar plot shows a consistent increase in sales from January to March, followed by a slight dip in April, and then a significant surge in May. Overall, there's a positive trend in sales."

2. Scatter Plot:

- *Question:* "What insights can you gather from the scatter plot comparing temperature and ice cream sales?"
- *Answer:* "Looking at the scatter plot, it appears there is a positive correlation between temperature and ice cream sales. As temperature increases, there is a corresponding rise in ice cream sales."

3. Line Plot:

- *Question:* "Explain the patterns observed in the line plot representing stock prices over the last six months."
- *Answer:* "In the line plot, we see some volatility in the stock prices in the initial months, with a sharp increase in value around the third month. However, there's a gradual decline afterward, suggesting fluctuating market conditions."

4. Pie Chart:

- *Question:* "What proportions of different product categories are represented in the pie chart?"
- *Answer:* "The pie chart indicates that the majority of sales come from the Electronics category, contributing 45%, followed by Clothing at 30%, and Home Goods at 25%. Electronics is the dominant category."

5. Box Plot:

- *Question:* "What can you infer from the box plot comparing the delivery times of two different courier services?"
- *Answer:* "The box plot suggests that Courier A generally has a more consistent and faster delivery time compared to Courier B. Courier B shows a wider range, indicating more variability in delivery times."

Program 7: Implement different data structures in R (Vectors, Lists, DataFrames)

AIM: Illustration of data structures in R

DESCRIPTION:

Creating Vectors:

- You can create a vector in R using the `c()` function, which stands for "combine" or "concatenate".
For example:

Numeric vector

```
numeric_vector <- c(1, 2, 3, 4, 5)
```

Character vector

```
character_vector <- c("apple", "banana", "orange")
```

Logical vector

```
logical_vector <- c(TRUE, FALSE, TRUE)
```

Vector Operations:

- You can perform various operations on vectors such as addition, subtraction, multiplication, division, and more. These operations are applied element-wise.

Example operations

```
x <- c(1, 2, 3)
```

```
y <- c(4, 5, 6)
```

Addition

```
result_addition <- x + y # Result: c(5, 7, 9)
```

Subtraction

```
result_subtraction <- x - y # Result: c(-3, -3, -3)
```

Multiplication

```
result_multiplication <- x * y # Result: c(4, 10, 18)
```

Division

```
result_division <- y / x # Result: c(4, 2.5, 2)
```

Vector Indexing:

- You can access elements of a vector using indexing. Indexing in R starts from 1.

Accessing elements of a vector

```
x <- c(10, 20, 30, 40, 50)
```

Accessing the second element

```
second_element <- x[2] # Result: 20
```

Accessing elements 3 to 5

```
elements_3_to_5 <- x[3:5] # Result: c(30, 40, 50)
```

Vector Functions:

- R provides various functions to work with vectors, such as `length()`, `sum()`, `mean()`, `min()`, `max()`, `sort()`, `rev()`, etc. These functions allow you to perform calculations and manipulations on vectors efficiently.

Example functions

```
x <- c(10, 20, 30, 40, 50)
```

Length of vector

```
vector_length <- length(x) # Result: 5
```

Sum of elements

```
sum_of_elements <- sum(x) # Result: 150
```

Mean of elements

```
mean_of_elements <- mean(x) # Result: 30
```

Minimum value

```
min_value <- min(x) # Result: 10
```

Maximum value

```
max_value <- max(x) # Result: 50
```

Sorting elements

```
sorted_x <- sort(x) # Result: c(10, 20, 30, 40, 50)
```

Reversing elements

```
reversed_x <- rev(x) # Result: c(50, 40, 30, 20, 10)
```

Vector Recycling Rule:

- In R, if you perform an operation on vectors of different lengths, R will recycle the shorter vector to match the length of the longer vector. This recycling behavior might not always yield desired results, so it's important to understand how it works.

LIST IN R

Creating Lists:

- You can create a list in R using the `list()` function. Inside the function, you can specify the elements you want to include in the list.

Creating a list with different types of elements

```
my_list <- list(name = "John", age = 30, scores = c(85, 90, 95), is_student = TRUE)
```

Accessing Elements:

- You can access elements of a list using double square brackets `[[ ]]` or the dollar sign `$`.

Accessing elements of a list

```
my_list <- list(name = "John", age = 30, scores = c(85, 90, 95), is_student = TRUE)
```

Accessing by index

```
name <- my_list[[1]] # "John"
```

Accessing by name

```
age <- my_list$age # 30
```

Nested Lists:

- Lists can contain other lists, allowing for nested structures.

Creating a nested list

```
nested_list <- list(
  name = "Alice",
  info = list(age = 25, city = "New York"),
  scores = c(80, 85, 90)
)
```

Manipulating Lists:

- You can add elements to a list using the concatenation operator `c()`.

Adding elements to a list

```
my_list <- list(name = "John", age = 30)
```

```
my_list <- c(my_list, scores = c(85, 90, 95), is_student = TRUE)
```

You can remove elements using the **NULL** keyword.

Removing elements from a list

```
my_list <- list(name = "John", age = 30, scores = c(85, 90, 95), is_student = TRUE)
```

```
my_list$scores <- NULL
```

Iterating Over Lists:

- You can iterate over elements of a list using loops or functional programming constructs like `lapply()` or `sapply()`.

Iterating over elements of a list

```
my_list <- list(name = "John", age = 30, scores = c(85, 90, 95), is_student = TRUE)
```

Using a for loop

```
for (element in my_list) {
  print(element)
}
```

Using lapply

```
lapply(my_list, print)
```

DATA FRAMES IN R:

Creating DataFrames:

- You can create a DataFrame using the `data.frame()` function. Each column of the DataFrame can be of a different data type (numeric, character, factor, etc.).

Creating a DataFrame

```
df <- data.frame(
  Name = c("John", "Alice", "Bob"),
  Age = c(25, 30, 35),
  Score = c(80, 85, 90),
  Passed = c(TRUE, TRUE, FALSE)
)
```

Accessing Elements:

- You can access elements of a DataFrame using row and column indexes, or by column names.

Accessing elements of a DataFrame

```
df <- data.frame(
  Name = c("John", "Alice", "Bob"),
  Age = c(25, 30, 35),
  Score = c(80, 85, 90),
  Passed = c(TRUE, TRUE, FALSE)
)
```

Accessing by row and column index

```
element <- df[2, 3] # Element at row 2, column 3
```

```
# Accessing by column name
```

```
column <- df$Age # Age column
```

Manipulating DataFrames:

- You can add new columns to a DataFrame or modify existing ones.

```
# Adding a new column
```

```
df$Grade <- c("A", "B", "C")
```

```
# Modifying existing column
```

```
df$Passed <- c(TRUE, FALSE, TRUE)
```

You can remove columns using the `NULL` keyword or the `subset()` function.

```
# Removing a column
```

```
df$Grade <- NULL
```

```
# Removing rows based on condition
```

```
df <- subset(df, Age > 25)
```

Summarizing DataFrames:

- R provides functions like `summary()`, `str()`, and `head()` to summarize and explore the contents of a DataFrame.

```
# Summary of the DataFrame
```

```
summary(df)
```

```
# Structure of the DataFrame
```

```
str(df)
```

```
# Displaying the first few rows of the DataFrame
```

```
head(df)
```

Merging and Concatenating DataFrames:

- You can merge two DataFrames based on common columns using functions like `merge()` or `dplyr::join()`.
- You can concatenate DataFrames vertically or horizontally using functions like `rbind()` and `cbind()`.

VIVA VOICE QUESTION & ANSWERS

1. **Question:** "What is the primary data structure used in R programming?"

- **Answer:** "The primary data structure in R programming is the DataFrame. It's a two-dimensional labeled data structure resembling a table in a relational database or a spreadsheet."

2. **Question:** "How can you create a vector in R?"

- **Answer:** "You can create a vector in R using the `c()` function, which stands for 'combine'. For example, `my_vector <- c(1, 2, 3, 4, 5)` creates a numeric vector."

3. **Question:** "What is the purpose of lists in R?"

- **Answer:** "Lists in R are versatile data structures that can hold elements of different types, such as vectors, matrices, data frames, or even other lists. They provide flexibility in organizing heterogeneous data."

4. **Question:** "Explain the concept of indexing in R vectors."

- **Answer:** "Indexing in R vectors allows you to access specific elements within a vector. Indexing starts from 1 in R. For example, `my_vector[3]` would access the third element of the vector."

5. **Question:** "How can you add a new column to a DataFrame in R?"

- **Answer:** "You can add a new column to a DataFrame in R by simply assigning values to it using the dollar sign notation. For example, `my_dataframe$new_column <- c(1, 2, 3)` adds a new column named 'new_column' with the specified values."

6. **Question:** "What function is used to remove elements from a list in R?"

- **Answer:** "To remove elements from a list in R, you can assign the `NULL` value to the element you want to remove. For instance, `my_list$element_name <- NULL` removes the specified element from the list."

7. **Question:** "How do you concatenate two DataFrames vertically in R?"

- **Answer:** "In R, you can concatenate two DataFrames vertically using the `rbind()` function. It stacks the rows of the second DataFrame below the rows of the first DataFrame."

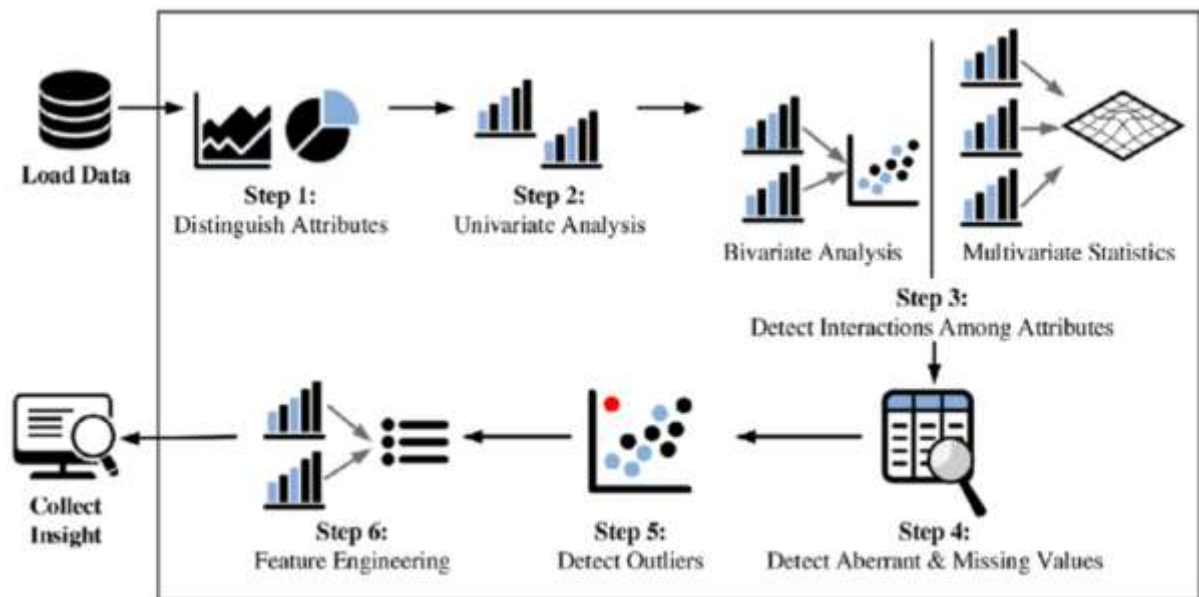
Program 8: Write an R program to read a csv file and analyze the data in the file using EDA (ExplorativeData Analysis) techniques.

DESCRIPTION:

Bike Buyers Dataset - EDA

Exploratory Data Analysis refers to the critical process of performing initial investigations on data so as to discover patterns, to spot anomalies, to test hypothesis and to check assumptions with the help of summary statistics and graphical representations.

It is a good practice to understand the data first and try to gather as many insights from it. EDA is all about making sense of data in hand, before getting them dirty with it.



SOURCE CODE:

In [1]:

```
library('ggvis')
library('tidyverse')
library('ggplot2')
```

— Attaching packages — tidyverse 1.3.0 —

```
✓ ggplot2 3.3.2    ✓ purrr  0.3.4
✓ tibble  3.0.3    ✓ dplyr  1.0.2
✓ tidyr   1.1.2    ✓ stringr 1.4.0
✓ readr   1.3.1    ✓ forcats 0.5.0
```

— Conflicts — tidyverse_conflicts() —

```
✗ dplyr::filter() masks stats::filter()
✗ dplyr::lag()    masks stats::lag()
```

Read the dataset and get high level understanding

```
In [2]: bike_buyers = read.csv('../input/bike-buyers/bike_buyers.csv', header=T, na.strings='')
head(bike_buyers)
```

A data.frame: 6 × 13

	ID	Marital.Status	Gender	Income	Children	Education	Occupation	Home.Owner	Cars	Commute.Distance	Region	Age	Purchased.Bike
	<int>	<fct>	<fct>	<int>	<int>	<fct>	<fct>	<fct>	<int>	<fct>	<fct>	<int>	<fct>
1	12496	Married	Female	40000	1	Bachelors	Skilled Manual	Yes	0	0-1 Miles	Europe	42	No
2	24107	Married	Male	30000	3	Partial College	Clerical	Yes	1	0-1 Miles	Europe	43	No
3	14177	Married	Male	80000	5	Partial College	Professional	No	2	2-5 Miles	Europe	60	No
4	24381	Single	NA	70000	0	Bachelors	Professional	Yes	1	5-10 Miles	Pacific	41	Yes
5	25597	Single	Male	30000	0	Bachelors	Clerical	No	0	0-1 Miles	Europe	36	Yes
6	13507	Married	Female	10000	2	Partial College	Manual	Yes	0	1-2 Miles	Europe	50	No

```
In [3]: class(bike_buyers)
```

'data.frame'

```
In [4]: str(bike_buyers)
```

```
'data.frame': 1000 obs. of 13 variables:
 $ ID      : int  12496 24107 14177 24381 25597 13507 27974 19364 22155 19288 ...
 $ Marital.Status : Factor w/ 2 levels "Married","Single": 1 1 1 2 2 1 2 1 NA 1 ...
 $ Gender      : Factor w/ 2 levels "Female","Male": 1 2 2 NA 2 1 2 2 2 2 ...
 $ Income      : int  40000 30000 80000 70000 30000 10000 160000 40000 20000 NA ...
 $ Children    : int  1 3 5 0 0 2 2 1 2 2 ...
 $ Education   : Factor w/ 5 levels "Bachelors","Graduate Degree",...: 1 4 4 1 1 4 3 1 5 4 ...
 $ Occupation  : Factor w/ 5 levels "Clerical","Management",...: 5 1 4 4 1 3 2 5 1 3 ...
 $ Home.Owner  : Factor w/ 2 levels "No","Yes": 2 2 1 2 1 2 NA 2 2 2 ...
 $ Cars        : int  0 1 2 1 0 0 4 0 2 1 ...
 $ Commute.Distance: Factor w/ 5 levels "0-1 Miles","1-2 Miles",...: 1 1 4 5 1 2 1 1 5 1 ...
 $ Region      : Factor w/ 3 levels "Europe","North America",...: 1 1 1 3 1 1 3 1 3 1 ...
 $ Age         : int  42 43 60 41 36 50 33 43 58 NA ...
 $ Purchased.Bike : Factor w/ 2 levels "No","Yes": 1 1 1 2 2 1 2 2 1 2 ...
```

```
In [5]: summary(bike_buyers)
```

	ID	Marital.Status	Gender	Income	Children
Min.	:11000	Married:535	Female:489	Min. : 10000	Min. :0.00
1st Qu.:	15291	Single :458	Male :500	1st Qu.: 30000	1st Qu.:0.00
Median :	19744	NA's : 7	NA's : 11	Median : 60000	Median :2.00
Mean :	19966			Mean : 56268	Mean :1.91
3rd Qu.:	24471			3rd Qu.: 70000	3rd Qu.:3.00
Max. :	29447			Max. :170000	Max. :5.00
				NA's :6	NA's :8

```
In [6]: levels(bike_buyers$Gender)
```

'Female' · 'Male'

Assign factors to string values

```
In [7]: bike_buyers$Marital.Status <- as.factor(bike_buyers$Marital.Status)
bike_buyers$Gender <- as.factor(bike_buyers$Gender)
bike_buyers$Home.Owner <- as.factor(bike_buyers$Home.Owner)
bike_buyers$Purchased.Bike <- as.factor(bike_buyers$Purchased.Bike)
```

```
In [8]: str(bike_buyers)
```

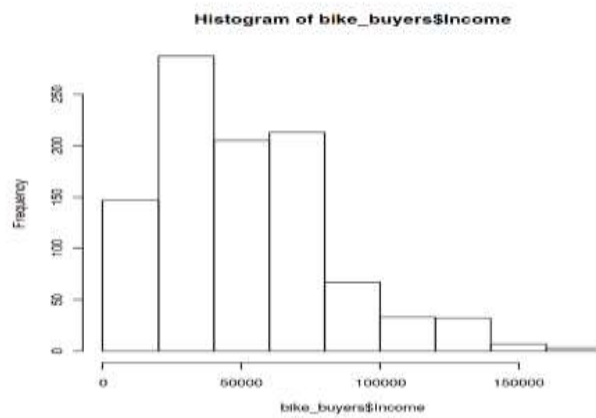
```
In [9]: colSums(is.na(bike_buyers))
```

**ID: 0 Marital.Status: 7 Gender: 11 Income: 6 Children: 8 Education: 0 Occupation: 0 Home.Owner: 4 Cars: 9
Purchased.Bike: 0**

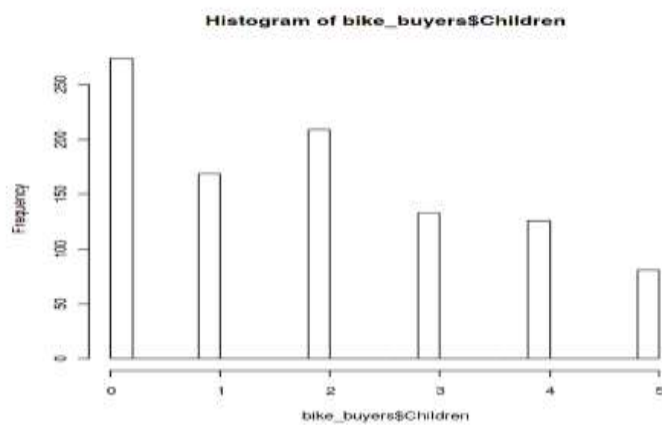
```
In [10]: summary(bike_buyers)
```

ID	Marital.Status	Gender	Income	Children
Min. :11000	Married:535	Female:489	Min. : 10000	Min. :0.00
1st Qu.:15291	Single :458	Male :500	1st Qu.: 30000	1st Qu.:0.00
Median :19744	NA's : 7	NA's : 11	Median : 60000	Median :2.00
Mean :19966			Mean : 56268	Mean :1.91
3rd Qu.:24471			3rd Qu.: 70000	3rd Qu.:3.00
Max. :29447			Max. :170000	Max. :5.00
			NA's :6	NA's :8

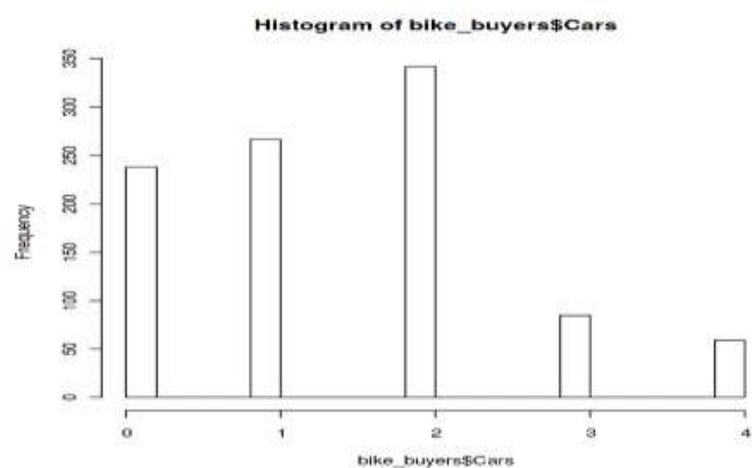
```
In [11]: hist(bike_buyers$Income)
```



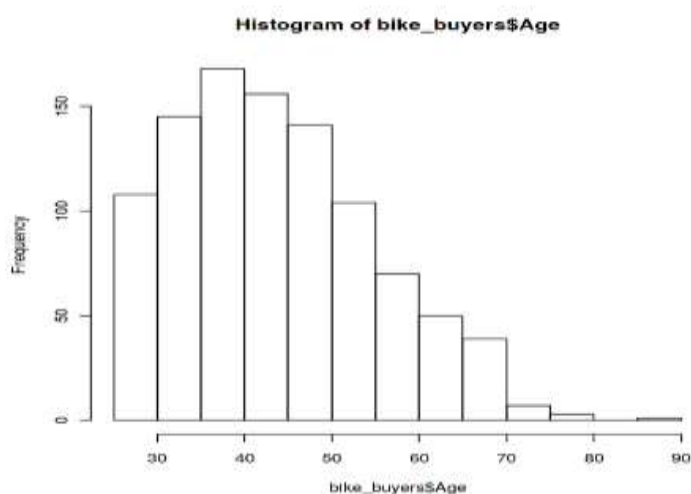
```
In [12]: hist(bike_buyers$Children, breaks = 20)
```



```
In [13]: hist(bike_buyers$Cars, breaks = 15)
```



```
In [14]: hist(bike_buyers$Age)
```



Dealing with NA values

Since, the distribution of Income and Age is left-skewed. We will impute median values

```
In [15]: median(na.omit((bike_buyers$Income)))  
median(na.omit((bike_buyers$Age)))
```

60000

43

```
In [16]: bike_buyers_clean <- bike_buyers  
colSums(is.na(bike_buyers_clean))
```

ID: 0 Marital.Status: 7 Gender: 11 Income: 6 Children: 8 Education: 0 Occupation: 0 Home.Owner: 4 Cars: 9

```
In [17]: # Income replaced with Median  
bike_buyers_clean$Income[is.na(bike_buyers_clean$Income)] <-  
  median(na.omit((bike_buyers$Income)))  
  
# Age replaced with Median  
bike_buyers_clean$Age[is.na(bike_buyers_clean$Age)] <-  
  median(na.omit((bike_buyers$Age)))  
  
colSums(is.na(bike_buyers_clean))
```

ID: 0 Marital.Status: 7 Gender: 11 Income: 0 Children: 8 Education: 0 Occupation: 0 Home.Owner: 4 Cars: 9

VIVA VOICE QUESTION & ANSWERS:

1. What is Exploratory Data Analysis (EDA)?

Answer: Exploratory Data Analysis (EDA) is an approach to analyzing datasets to summarize their main characteristics, often using visual methods. It helps in understanding the data, detecting patterns, identifying outliers, and formulating hypotheses.

2. **What are some common techniques used in EDA?**

Answer: Common techniques used in EDA include summary statistics, histograms, box plots, scatter plots, correlation matrices, density plots, and data transformation methods like log transformation.

3. **How can you read a CSV file in R for exploratory data analysis?**

Answer: In R, you can use functions like `read.csv()` or `read_csv()` from the `readr` package to read CSV files. For example:

```
data <- read_csv("your_file.csv")
```

4. **How do you check the structure of a dataset in R?**

Answer: You can use the `str()` function in R to check the structure of a dataset, which provides information about the data types and structure of each variable.

5. **What are summary statistics, and how do you compute them in R?**

Answer: Summary statistics provide a concise summary of the main characteristics of a dataset, including measures like mean, median, standard deviation, minimum, maximum, and quartiles. In R, you can use the `summary()` function to compute summary statistics for a dataset.

6. **How do you handle missing values in R during exploratory data analysis?**

Answer: In R, you can use functions like `is.na()` to detect missing values and `na.omit()` to remove rows with missing values. You can also impute missing values using techniques such as mean imputation or median imputation.

7. **What are some common visualizations used in EDA, and how do you create them in R?**

Answer: Common visualizations in EDA include histograms, box plots, scatter plots, and correlation matrices. In R, you can create these visualizations using packages like `ggplot2` for creating customized plots with high flexibility and clarity.

8. **How can you identify outliers in a dataset using EDA techniques in R?**

Answer: Outliers can be identified visually using box plots, scatter plots, or histograms. In R, you can create box plots and scatter plots to visualize the distribution of data points and identify any points that lie far from the bulk of the data.

9. **What is the purpose of performing correlation analysis in EDA?**

Answer: Correlation analysis helps to identify the strength and direction of the relationship between two variables in a dataset. It is useful for understanding dependencies between variables and can help in feature selection and predictive modeling.

10. **How can you interpret a correlation matrix in R?**

Answer: In R, you can compute a correlation matrix using the `cor()` function. The correlation matrix displays the correlation coefficients between pairs of variables. A value close to 1 indicates a strong positive correlation, close to -1 indicates a strong negative correlation, and close to 0 indicates little to no linear relationship.

Program9: Write an Rprogram to illustrate Linear Regression and Multilinear Regression considering suitable CSV file

DESCRIPTION:

Linear Regression:

The example of an R program that illustrates linear regression using a suitable CSV file. In this example, I'll use the `mtcars` dataset that comes with R. The goal is to predict the miles per gallon (`mpg`) of cars based on their horsepower (`hp`). You can replace it with your own dataset.

This program performs the following steps:

1. Loads the `ggplot2` library for visualization.
2. Loads the `mtcars` dataset.
3. Displays the first few rows of the dataset.
4. Performs linear regression using the `lm()` function.
5. Prints a summary of the linear regression model.
6. Visualizes the linear regression line along with the data using `ggplot2`.
7. Predicts the miles per gallon for new data points (horsepower values 100, 150, and 200).

You can replace the dataset with your own CSV file and adjust the variables accordingly. The `lm()` function is used to fit a linear regression model, and `predict()` can be used to make predictions for new data points.

SOURCE CODE:

```
# Load required libraries
```

```
library(ggplot2)
```

```
# Load the mtcars dataset
```

```
data(mtcars)
```

```
# Display the first few rows of the dataset
```

```
print(head(mtcars))
```

```
# Perform linear regression
```

```
linear_model <- lm(mpg ~ hp, data = mtcars)
```

```
# Summary of the linear regression model
```

```
summary(linear_model)
```

```
# Visualize the linear regression line
```

```
ggplot(mtcars, aes(x = hp, y = mpg)) +
```

```
  geom_point() +
```

```
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
```

```
labs(title = "Linear Regression: mpg vs hp",
```

```
  x = "Horsepower (hp)",
```

```
  y = "Miles per Gallon (mpg)")
```

```
# Predicting mpg for new data
```

```
new_data <- data.frame(hp = c(100, 150, 200))
```

```
predicted_mpg <- predict(linear_model, newdata = new_data)
```

```
print(predicted_mpg)
```

OUTPUT:

horsepower values 100, 150, and 200

VIVA VOICE QUESTIONS AND ANSWERS

1. **What is linear regression?**

Answer: Linear regression is a statistical method used to model the relationship between a dependent variable and one or more independent variables. It assumes a linear relationship between the independent and dependent variables.

2. **How is linear regression represented mathematically?**

Answer: The linear regression equation for a simple linear regression model with one independent variable can be represented as: $y = \beta_0 + \beta_1 x + \epsilon$, where y is the dependent variable, x is the independent variable, β_0 is the intercept, β_1 is the slope coefficient, and ϵ represents the error term.

3. **How do you fit a linear regression model in R?**

Answer: In R, you can fit a linear regression model using the `lm()` function. For example:
`model <- lm(dependent_variable ~ independent_variable, data = your_data)`

4. **What does the `summary()` function in R provide for a linear regression model?**

Answer: The `summary()` function provides a summary of the linear regression model, including coefficients, standard errors, t-values, p-values, and R-squared values.

5. **How do you interpret the coefficients in a linear regression model?**

Answer: The intercept (β_0) represents the value of the dependent variable when all independent variables are zero. The slope coefficient (β_1) represents the change in the dependent variable for a one-unit change in the independent variable, holding other variables constant.

6. **What is the R-squared value, and what does it indicate in linear regression?**

Answer: The R-squared value is a measure of how well the independent variables explain the variation in the dependent variable. It indicates the proportion of the variance in the dependent variable that is predictable from the independent variables. A higher R-squared value indicates a better fit of the model to the data.

7. **How do you make predictions using a linear regression model in R?**

Answer: You can make predictions using the `predict()` function in R. For example:
`predicted_values <- predict(model, newdata = new_data)`

8. **How can you assess the goodness of fit of a linear regression model in R?**

Answer: In addition to the R-squared value, you can assess the goodness of fit using diagnostic plots such as residual plots, QQ plots, and leverage plots.

9. **What are some assumptions of linear regression?**

Answer: Some assumptions of linear regression include linearity, independence of errors, constant variance of errors (homoscedasticity), and normality of errors.

10. **Can linear regression handle categorical predictors?**

Answer: Yes, linear regression can handle categorical predictors by encoding them as dummy variables or using techniques like ANOVA. In R, factors can be used to represent categorical variables.

Multiple Linear Regression:

example of an R program that illustrates multiple linear regression using a suitable CSV file. In this example, we'll use a CSV file named `cars.csv` containing data about car prices, their mileage, and the number of years they've been in use.

Explanation of the code:

1. We load the necessary libraries: `ggplot2` for visualization and `readr` for reading the CSV file.
2. We read the CSV file named `cars.csv` using the `read_csv()` function.
3. We display the first few rows of the dataset to get a sense of its structure.
4. We perform multiple linear regression using the `lm()` function, with `Price` as the dependent variable and `Mileage` and `Age` as independent variables.
5. We print a summary of the multiple linear regression model, which includes coefficients, standard errors, t-values, p-values, and R-squared values.
6. We visualize the linear regression line using `ggplot2`, with mileage on the x-axis, price on the y-axis, and different colors representing the age of the cars.
7. We predict the price for new data points using the `predict()` function, providing values for mileage and age.

Make sure to replace `"cars.csv"` with the correct path to your CSV file containing the relevant data. Also, adjust the column names according to your dataset. This code assumes that your CSV file has three columns: `Price`, `Mileage`, and `Age`, where `Price` is the dependent variable, and `Mileage` and `Age` are independent variables.

SOURCE CODE:

```
# Load required libraries
```

```
library(ggplot2)
```

```
library(readr)
```

```
# Read the CSV file
```

```
data <- read_csv("cars.csv") # Make sure to replace "cars.csv" with your CSV file path
```

```
# Display the first few rows of the dataset
```

```
print(head(data))
```

```
# Perform multiple linear regression
```

```
linear_model <- lm(Price ~ Mileage + Age, data = data)
```

```
# Summary of the multiple linear regression model
```

```
summary(linear_model)
```

```
# Visualize the linear regression line
```

```
ggplot(data, aes(x = Mileage, y = Price, color = Age)) +
```

```
geom_point() +  
geom_smooth(method = "lm", se = FALSE, aes(group = 1)) +  
labs(title = "Multiple Linear Regression: Price vs Mileage by Age",  
      x = "Mileage",  
      y = "Price",  
      color = "Age")  
  
# Predicting Price for new data  
new_data <- data.frame(Mileage = c(5000, 10000, 15000),  
                       Age = c(1, 2, 3)) # Assuming the age of the cars  
predicted_price <- predict(linear_model, newdata = new_data)  
print(predicted_price)
```

OUTPUT:

Predicts sales for new advertising and price values using the trained model.

Additional Experiments:

Program 10: What is the purpose of the `apply()` function in R? Can you explain how it works and provide an example?

DESCRIPTION:

The `apply()` function in R is used to apply a function (usually a user-defined or built-in function) over the margins of an array (e.g., matrix or array). It is particularly useful for performing operations on rows or columns of matrices or data frames. The general syntax of the `apply()` function is:

```
apply(X, MARGIN, FUN, ...)
```

- **x**: The array (matrix or data frame) over which the function will be applied.
- **MARGIN**: An integer vector indicating which margins should be 'retained'. Typically, **MARGIN** = 1 indicates rows, **MARGIN** = 2 indicates columns, and **MARGIN** = c(1, 2) indicates both rows and columns.
- **FUN**: The function to be applied. It can be a built-in function or a custom-defined function.
- **...**: Additional arguments to be passed to the function.

Here's an example to illustrate how the `apply()` function works:

Suppose we have a matrix `mat` representing students' scores on three exams:

```
mat <- matrix(c(80, 90, 75, 85, 70, 95), nrow = 3, byrow = TRUE)
colnames(mat) <- c("Exam1", "Exam2", "Exam3")
rownames(mat) <- c("Student1", "Student2", "Student3")
```

Now, if we want to calculate the mean score for each student, we can use the `apply()` function as follows:

```
# Calculate mean score for each student (along rows)
mean_scores <- apply(mat, MARGIN = 1, FUN = mean)
print(mean_scores)
```

1

Output:

```
Student1 Student2 Student3
 81.66667  83.33333  86.66667 83.33333 86.66667
```

In this example, we applied the `mean()` function over the rows (specified by **MARGIN** = 1) of the `mat` matrix, resulting in the mean score for each student. This is just one application of the `apply()` function, which demonstrates its versatility in R programming.

Scheme of Evaluation:

Internal Assessment

S. No.	Assessment of work	Evaluation in Marks
1	Lab observation-day to day work	20
2	Viva Voice	10
3	Internal laboratory Test	10
Total Marks		25

External Assessment

S. No.	Assessment of work	Evaluation in Marks
1	Write up	25
2	Experimentation	25
3	Viva	10
Total Marks		60

References:

1. RProgrammingforDataSciencebyRogerD.Peng
2. TheArtofRProgrammingbyNormanMatloffCengageLearningIndia.
3. HadleyWickham,GarrettGrolemund,RforDataScience:Import,Tidy,Transform,Visualize, and Model Data 1st Edition, O'Reilly
4. TilmanM.Davies,ThebookofRafirstcourseinprogrammingandstatistics,nostarchpres